

1. Geração de Códigos do Projeto

Essa seção apresenta as interações do programador para a geração do código do projeto. Nesse contexto, foi seguido um fluxo real de desenvolvimento. É importante destacar que nessas interações não solicitou-se a geração de códigos com acessibilidade.

1.1 Página inicial

Data de Implementação

10/12/2023

Comando de solicitação

1. Criar 2 divs, sendo 1 para título e a segunda conteúdo. O conteúdo deve conter 4 parágrafos, sendo o parágrafo 2 e 3 seguidos de lista.
2. Refatorar a div pai com o padrão de card do tailwind.
3. Formatar o card como se fosse uma página do word mantendo o padrão do tailwind.

Cenário

Início

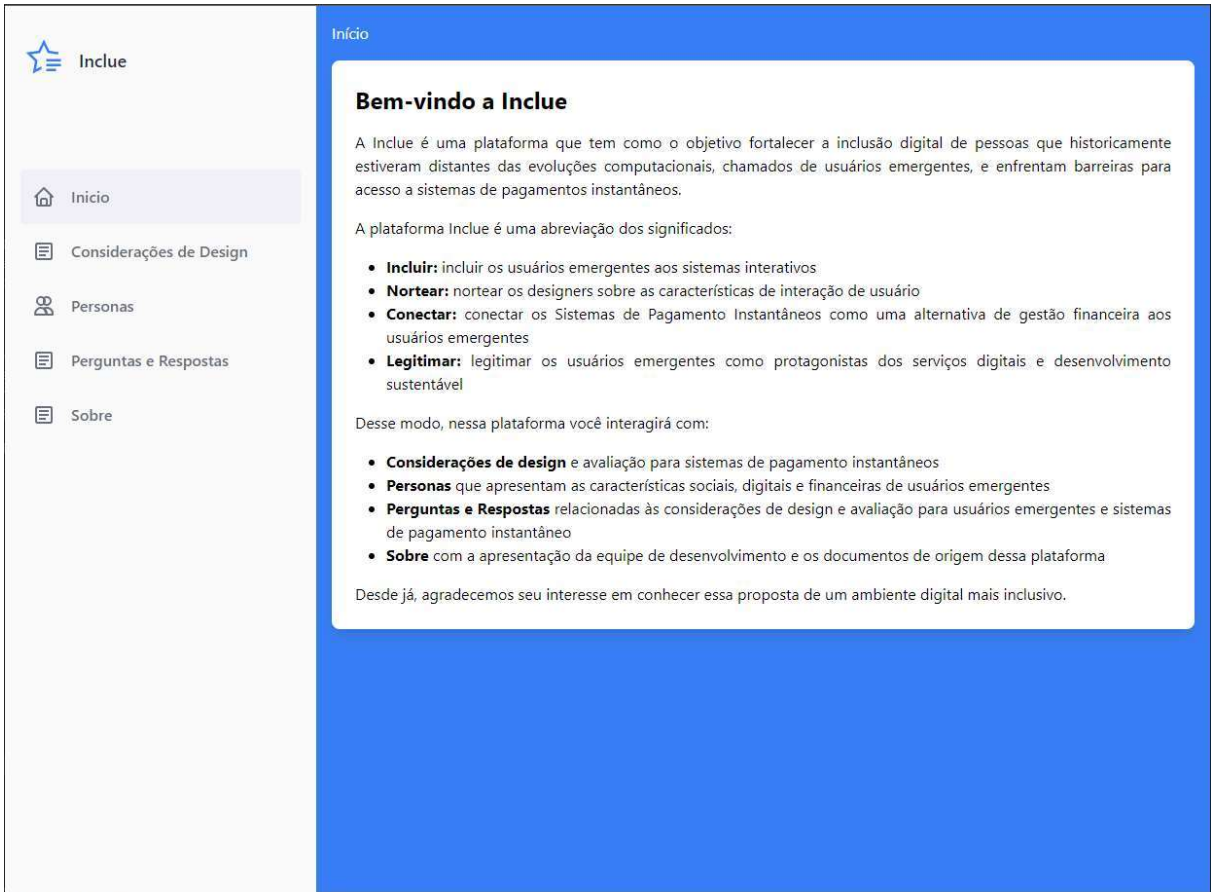
Evidência

Resultado do comando 1

```
src > pages > TS index.tsx > Home
1  import Layout from "../components/Layout";
2
3  export default function Home() {
4    return (
5      <Layout title="Início">
6        <div className="bg-white rounded-lg shadow-lg p-6">
7          <div>
8            <h1>Título</h1>
9          </div>
10         <div>
11           <p>Conteúdo</p>
12           <p>Parágrafo 2</p>
13           <ul>
14             <li>Item 1</li>
15             <li>Item 2</li>
16             <li>Item 3</li>
17           </ul>
18           <p>Parágrafo 3</p>
19           <ul>
20             <li>Item 4</li>
21             <li>Item 5</li>
22             <li>Item 6</li>
23           </ul>
24           <p>Parágrafo 4</p>
25         </div>
26       </div>
27     </Layout>
28   )
29 }
30
```

Resultados do comando 2 e 3

```
src > pages > TS indextsx > Home
1 import Layout from "../../components/Layout";
2
3 export default function Home() {
4   return (
5     <Layout title="Início">
6       <div className="bg-white rounded-lg shadow-lg p-6">
7         <div className="text-justify">
8           <h1 className="text-2xl font-bold mb-4">Bem-vindo a Incluir</h1>
9         </div>
10        <div className="text-justify">
11          <p className="mb-4">A Incluir é uma plataforma que tem como o objetivo fortalecer a inclusão digital de pessoas que historicamente
12          <p className="mb-4">A plataforma Incluir é uma abreviação dos significados:</p>
13          <ul className="list-disc ml-8 mb-4">
14            <li><strong>Incluir:</strong> incluir os usuários emergentes aos sistemas interativos</li>
15            <li><strong>Nortear:</strong> nortear os designers sobre as características de interação de usuário</li>
16            <li><strong>Conectar:</strong> conectar os Sistemas de Pagamento Instantâneos como uma alternativa de gestão financeira aos
17            <li><strong>Legitimar:</strong> legitimar os usuários emergentes como protagonistas dos serviços digitais e desenvolvimento
18          </ul>
19          <p className="mb-4">Desse modo, nessa plataforma você interagirá com:</p>
20          <ul className="list-disc ml-8 mb-4">
21            <li><strong>Considerações de design</strong> e avaliação para sistemas de pagamento instantâneos</li>
22            <li><strong>Personas</strong> que apresentam as características sociais, digitais e financeiras de usuários emergentes</li>
23            <li><strong>Perguntas e Respostas</strong> relacionadas às considerações de design e avaliação para usuários emergentes e sistemas
24            <li><strong>Sobre</strong> com a apresentação da equipe de desenvolvimento e os documentos de origem dessa plataforma</li>
25          </ul>
26          <p>Desde já, agradecemos seu interesse em conhecer essa proposta de um ambiente digital mais inclusivo.</p>
27        </div>
28      </div>
29    </Layout>
30  )
31 }
32
```



Observação

Tentei refatorar o CSS do título usando copilot e ele não me deu apoio necessário, dessa forma precisei realizar a implementação de forma manual após o comando 2, ainda após o comando 2 fiz a cópia dos conteúdos do figma e executei o comando 3 que deu o resultado final de tela formatada conforme um arquivo do word.

1.2 Componente para considerações de design

Data de Implementação

10/12/2023

Comando de solicitação

1. Implementar na div o mesmo padrão de card do tailwind
2. div com fundo azul turquesa e fonte branca com bordar arredondadas e negrito

Cenário

Considerações de Design > Componente de card para tela de busca

Evidência

```
src > components > basics > TS ConsiderationCard.tsx > ConsiderationCard
1 | import Link from "next/link"
2 |
3 | interface ConsiderationCardProps {
4 |   id: number
5 |   title: string
6 |   category: string
7 |   content: string
8 | }
9 |
10 | export default function ConsiderationCard(props: ConsiderationCardProps) {
11 |   return (
12 |     <div className="flex flex-col justify-center items-center">
13 |       <div className="font-bold text-xl mb-2">
14 |         {props.title}
15 |       </div>
16 |       <div className="bg-primary text-justify text-white rounded-lg">
17 |         {props.category}
18 |       </div>
19 |       <div className="text-justify">
20 |         {props.content.substring(0, 100) + "..."}
21 |       </div>
22 |       <div className="text-blue-500 underline text-end">
23 |         <Link href={`/incluir/consideration/${props.id}`} >
24 |           Saiba mais
25 |         </Link>
26 |       </div>
27 |     </div>
28 |   )
29 | }
```

Observação

Iniciei o desenvolvimento criando a interface `ConsiderationCardProps` para conseguir pegar as informações de consideração em nosso componente.

Fiz a implementação das divs de forma manual para deixar o componente segregado com linhas para receber o nosso objeto de consideração e após isso executei o comando 1, para refatorar a página conforme um card utilizando tailwind.

Dessa forma o copilot gerou o estilo de card porém ainda precisei fazer alguns ajustes manuais para o estilo da categoria e o estilo do saiba mais, o estilo da categoria foi refatorado utilizando o comando 3 porém precisei alterar as cores para o padrão que já estava utilizando. Na implementação do componente `ConsiderationCard` o Copilot gerou o link corretamente, porém mandou o caminho para outra tela que seria a de listagens de considerações (`ConsiderationsPage`) e não a tela de considerações (`ConsierationPage`).

```
src > components > basics > TS ConsiderationCard.tsx > ConsiderationCard
1 | import Link from "next/link"
2 |
3 | interface ConsiderationCardProps {
4 |   id: number
5 |   title: string
6 |   category: string
7 |   content: string
8 | }
9 |
10 | export default function ConsiderationCard(props: ConsiderationCardProps) {
11 |   return (
12 |     <div className="flex flex-col justify-center items-center">
13 |       <div className="font-bold text-xl mb-2">
14 |         {props.title}
15 |       </div>
16 |       <div className="text-justify">
17 |         {props.category}
18 |       </div>
19 |       <div className="text-justify">
20 |         {props.content}
21 |       </div>
22 |       <Link href={`/inclue/considerations/${props.id}`}>
23 |     </div>
24 |   )
25 | }
```

1.3 Página de listagem das considerações de design

Data de Implementação

10/12/2023

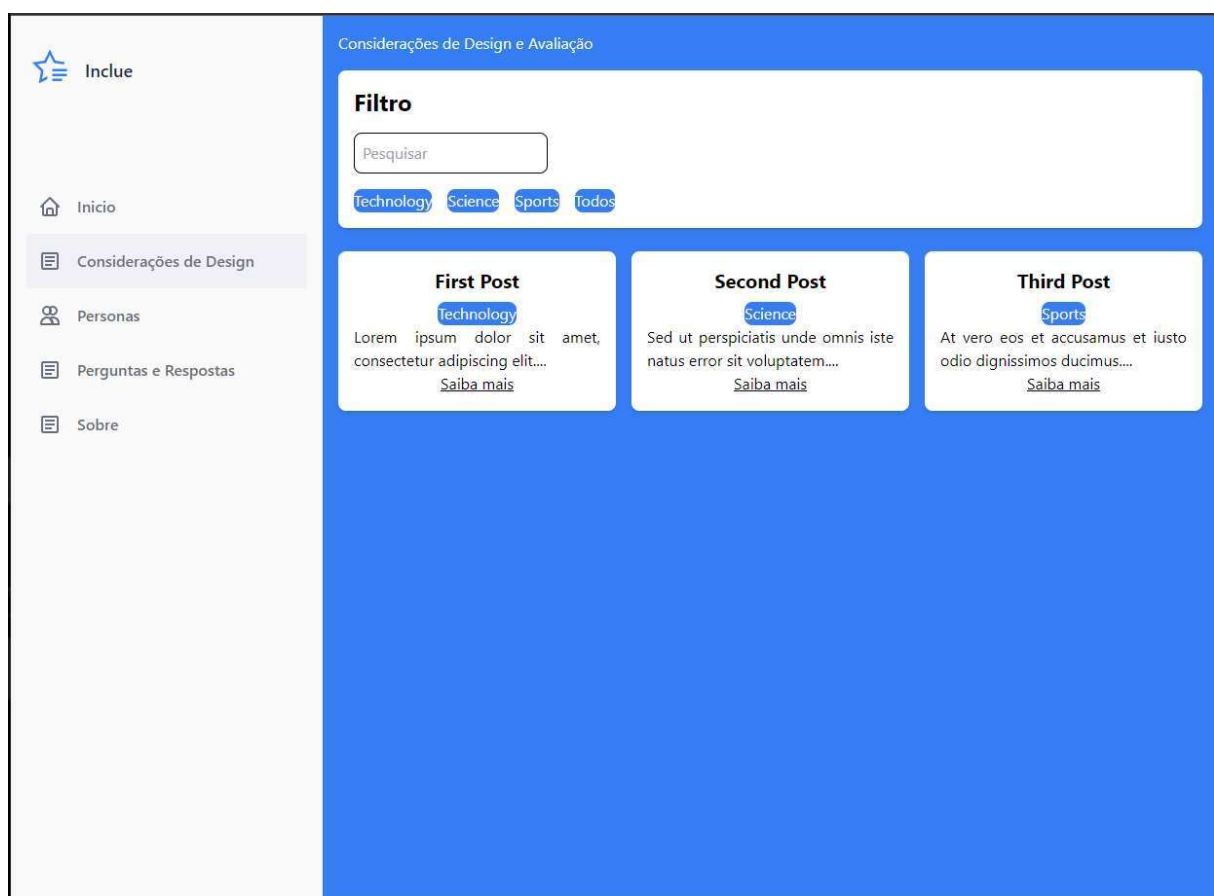
Comando de solicitação

1. Implementar na div o mesmo padrão de card do tailwind
2. Refatorar código para implementar uma campo de busca que filtre parte do texto de consideration.title ou consideration.content

Cenário

Considerações de Design > Busca com Pesquisa

Evidência



```

src > pages > include > TS considerations.tsx > ConsiderationsPage > map() callback
1  import Layout from "@components/Layout";
2  import { considerationsByCategory } from "../../db/considerationsByCategory";
3  import ConsiderationCard from "@components/basics/ConsiderationCard";
4
5
6  export default function ConsiderationsPage() {
7
8      //var filter = dummyCategories[0]
9
10     var filter = ""
11
12     return (
13         <Layout title="Considerações de Design e Avaliação">
14
15             <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4">
16                 {considerationsByCategory(filter).map((consideration) => {
17                     return (
18                         <div className="bg-white rounded-lg shadow-md p-4">
19                             <ConsiderationCard
20                                 key={consideration.id}
21                                 id={consideration.id}
22                                 title={consideration.title}
23                                 category={consideration.category}
24                                 content={consideration.content} />
25                         </div>
26                     );
27                 })}
28             </div>
29         </Layout>
30     )
31 }
32

```

Observação

Inicialmente criei a div e o código para fazer a listagem de forma manual utilizando o componente de Layout, após a criação da div principal criei uma div para cada consideração que seria listada colocando o componente ConsiderationCard nela com seus respectivos parâmetros do objeto.

Ao executar o comando 1 ele carregou o padrão de cards que estamos utilizando na página inicial e já responsivo nele inclui o nosso componente ConsiderationCard, porém precisei fazer algumas refatorações nele que citei nas observações do item 1.2.

1.4 Página de detalhe das considerações de design

Data de Implementação

10/12/2023

Comando de solicitação

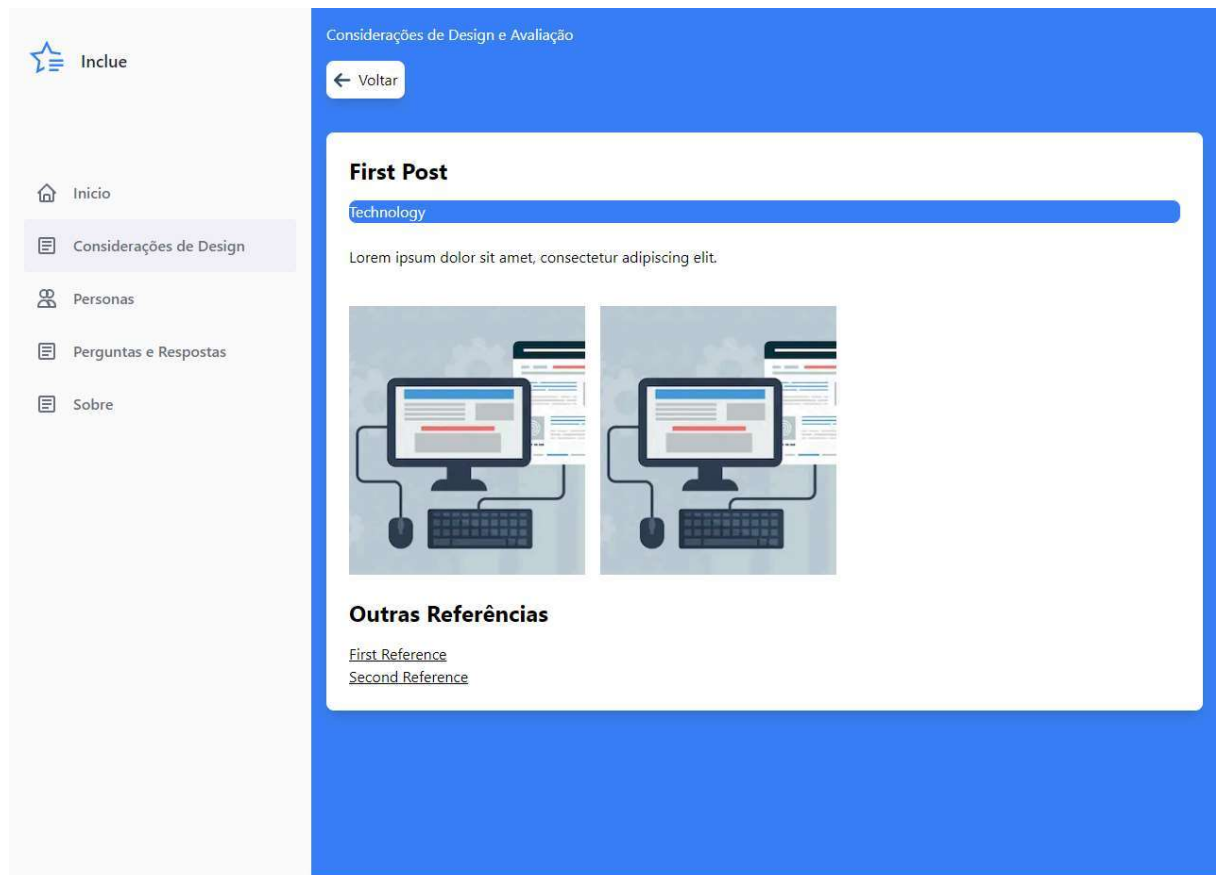
1. Implementar na div o mesmo padrão de card do tailwind

2. Refatorar a div do link para ficar com underline e cor azul

Cenário

Considerações de Design > Detalhe

Evidência



Div do botão voltar

```

src > pages > include > TS consideration.tsx > ConsiderationDetailPage
1  import Layout from "@components/Layout";
2  import Link from "next/link";
3  import { useRouter } from 'next/router'
4  import { dummyConsiderations } from "../../db/dummyConsiderations";
5  import Image from 'next/image'
6
7  export default function ConsiderationDetailPage() {
8
9      const { query } = useRouter()
10     var id = query.id
11     var consideration = dummyConsiderations.find((consideration) => consideration.id === parseInt(id))
12
13     return (
14         <Layout title="Considerações de Design e Avaliação">
15             <div className="text-justify">
16                 <Link className="flex" href="/include/considerations">
17                     <div className="flex rounded-lg shadow-lg p-2 mb-3 gap-2 items-center bg-white ">
18                         <svg width="18" height="15" viewBox="0 0 18 15" fill="none" xmlns="http://www.w3.org/2000/svg">
19                             <path d="M0.351562 6.60156L6.60156 0.351562C6.86198 0.117188 7.16146 0 7.5 0C7.83854 0 8.13802 0.117188 8.39844 0.351562L14.6016 6.60156C14.8619 6.86198 15 7.16146 15 7.5C15 7.83854 14.8619 8.13802 14.6016 8.39844L8.39844 14.6016C8.13802 14.8619 7.83854 15 7.5 15C7.16146 15 6.86198 14.8619 6.60156 14.6016L0.351562 8.39844C0.117188 8.13802 0 7.83854 0 7.5C0 7.16146 0.117188 6.86198 0.351562 6.60156Z" />
20                         </svg>
21                         Voltar
22                     </div>
23                 </Link>
24             </div>
25             <br />
26             <div>...
27         </div>
28     </Layout>
29 )
30 }

```

Div principal com título, categoria e conteúdo

```

src > pages > include > TS consideration.tsx > ConsiderationDetailPage
1  import Layout from "@components/Layout";
2  import Link from "next/link";
3  import { useRouter } from "next/router";
4  import { dummyConsiderations } from "../../db/dummyConsiderations";
5  import Image from "next/image";
6
7  export default function ConsiderationDetailPage() {
8
9      const { query } = useRouter()
10     var id = query.id
11     var consideration = dummyConsiderations.find((consideration) => consideration.id === parseInt(id))
12
13     return (
14         <Layout title="Considerações de Design e Avaliação">
15             <div className="text-justify">...
24         </div>
25         <br />
26         <div>
27             <div className="bg-white rounded-lg shadow-lg p-6">
28                 <div className="text-justify">
29                     <h1 className="text-2xl font-bold mb-4">{consideration?.title}</h1>
30                     <div className="bg-primary text-white rounded-lg">
31                         {consideration?.category}
32                     </div>
33                     <br />
34                     <p className="mb-4">{consideration?.content}</p>
35                 </div>
36             <br />
37             <div className="text-justify">...
51         </div>
52         <br />
53         <div className="text-justify">...
66     </div>
67     </div>
68 </div>
69 </Layout>
70 )
71 }

```

Div com a lista de imagens

```
src > pages > include > TS consideration.tsx > ConsiderationDetailPage
6
7 export default function ConsiderationDetailPage() {
8
9   const { query } = useRouter()
10  var id = query.id
11  var consideration = dummyConsiderations.find((consideration) => consideration.id === parseInt(id))
12
13  return (
14    <Layout title="Considerações de Design e Avaliação">
15 > <div className="text-justify"> ...
24 </div>
25 <br />
26 <div>
27   <div className="bg-white rounded-lg shadow-lg p-6">
28 > <div className="text-justify"> ...
35 </div>
36 <br />
37 <div className="text-justify">
38   <ul className="flex space-x-4">
39     {consideration?.images.map((image) => {
40       return (
41         <li>
42           <Image src={image.url}
43             width={250}
44             height={250}
45             alt={image.text}
46           />
47         </li>
48       );
49     })}
50   </ul>
51 </div>
52 <br />
53 > <div className="text-justify"> ...
66 </div>
67 </div>
68 </Layout>
69
70
71 }
```

Div com outras referências

```
src > pages > include > TS consideration.tsx > ConsiderationDetailPage
6
7 export default function ConsiderationDetailPage() {
8
9   const { query } = useRouter()
10   var id = query.id
11   var consideration = dummyConsiderations.find((consideration) => consideration.id === parseInt(id))
12
13   return (
14     <Layout title="Considerações de Design e Avaliação">
15 > <div className="text-justify">...
24 </div>
25 <br />
26 <div>
27   <div className="bg-white rounded-lg shadow-lg p-6">
28 > <div className="text-justify">...
35 </div>
36 <br />
37 > <div className="text-justify">...
51 </div>
52 <br />
53 <div className="text-justify">
54   <h1 className="text-2xl font-bold mb-4">Outras Referências</h1>
55   <ol className="text-justify text-blue-500 underline">
56     {consideration?.references.map((reference) => {
57       return (
58         <li>
59           <Link href={reference.url} >
60             {reference.title}
61           </Link>
62         </li>
63       );
64     })}
65   </ol>
66 </div>
67 </div>
68 </div>
69 </Layout>
70
71 }
```

Observação

Iniciei o desenvolvimento da página de forma manual incluindo o componente de Layout com título e o Link voltar de forma manual para conseguir voltar para a página de busca de considerações.

Tentei refatorar o link voltar com copilot mas não tive sucesso e precisei implementar o estilo de forma manual.

No segundo momento gerei as divs para separar a estrutura da tela conforme as especificações do figma e fui colocando o conteúdo em divs separadas sem as imagens no primeiro momento. Após isso executei o comando 1 para refatorar a tela para se adequar ao padrão de cards que usei nas demais telas.

Também implementei a div com a listagem de imagens de forma manual copiando o mesmo estilo dos Cards para criar as colunas.

Ao implementar a tela de detalhe da consideração o Copilot sugeriu no componente imagem incluir o texto alternativo.

Fiz a cópia de forma manual do estilo definido na categoria para deixar no mesmo padrão em todas as telas.

Por último criei uma div para outras referências no formato de lista de forma manual e refatorei o link utilizando comando 3 para ficar com estilo de url a exibição.

Fiz uma refatoração manual para reduzir o número de divs geradas para facilitar a manutenção futura.

1.5 Componente de Personas

Data de Implementação

13/12/2023

Comando de solicitação

1. criar uma div de estilo card como foi feito em ConsiderantonCar.tsx para exibir PersonaCardProps com tailwind

Cenário

Personas > Componente para listagem

Evidência

```
src > components > basics > TS PersonaCard.tsx > PersonaCard
1
2 interface PersonaCardProps {
3   id: number
4   name: string
5   description: string
6   image: string
7   characteristics: []
8 }
9
10 export default function PersonaCard(props: PersonaCardProps) {
11   return (
12     <div className="bg-white rounded-lg shadow-md p-4">
13       <h2 className="text-xl font-bold mb-2">{props.name}</h2>
14       <p className="text-gray-500 mb-4">{props.description}</p>
15       <img src={props.image} alt={props.name} className="w-full mb-4" />
16       <ul>
17         {props.characteristics.map((characteristic, index) => (
18           <li key={index} className="text-gray-700">{characteristic}</li>
19         ))}
20       </ul>
21     </div>
22   )
23 }
```

Observação

Ao executar o comando 1 para montar o componente `PersonaCard` o Copilot gerou o seguinte código abaixo, dessa forma foi necessário refatorar o código gerado para usar somente o que eu precisava, observe o detalhe gerado para tag imagem para texto alternativo (alt).

```
src > components > basics > TS PersonaCard.tsx > PersonaCard > PersonaCard
1
2 interface PersonaCardProps {
3   id: number
4   name: string
5   description: string
6   image: string
7   characteristics: []
8 }
9
10 export default function PersonaCard(props: PersonaCardProps) {
11   return (
12     import React from 'react';
13
14     interface PersonaCardProps {
15       id: number;
16       name: string;
17       description: string;
18       image: string;
19       characteristics: [];
20     }
21
22     export default function PersonaCard(props: PersonaCardProps) {
23       return [
24         <div className="bg-white rounded-lg shadow-md p-4">
25           <h2 className="text-xl font-bold mb-2">{props.name}</h2>
26           <p className="text-gray-500 mb-4">{props.description}</p>
27           <img src={props.image} alt={props.name} className="w-full mb-4" />
28           <ul>
29             {props.characteristics.map((characteristic, index) => (
30               <li key={index} className="text-gray-700">{characteristic}</li>
31             ))}
32           </ul>
33         </div>
34       ];
35     }
36   )
37 }
```

Também foi necessário fazer um ajuste manual na propriedade `characteristics` colocando o tipo do array como `string`.

```
interface PersonaCardProps {
  id: number
  name: string
  description: string
  image: string
  characteristics: string[]
}
```

Precisei remover a `div` do componente para não ficar com `divs` duplicadas em relação a tela de personas e incluí o texto resumo das características conforme as especificações do figma.

```
src > components > basics > TS PersonaCard.tsx > PersonaCard
1
2 interface PersonaCardProps {
3   id: number
4   name: string
5   description: string
6   image: string
7   characteristics: string[]
8 }
9
10 export default function PersonaCard(props: PersonaCardProps) {
11   return (
12     <div>
13       <h2 className="text-xl font-bold mb-2">{props.name}</h2>
14       <p className="text-gray-500 mb-4">{props.description}</p>
15       <img src={props.image} alt={props.name} className="w-full mb-4" />
16       <h1 className="text-2xl font-bold mb-4">Resumo das Características</h1>
17       <ul>
18         {props.characteristics.map((characteristic, index) => (
19           <li key={index} className="text-gray-700">{characteristic}</li>
20         ))}
21       </ul>
22     </div>
23   )
24 }
```

1.6 Tela de Personas

Data de Implementação

13/12/2023

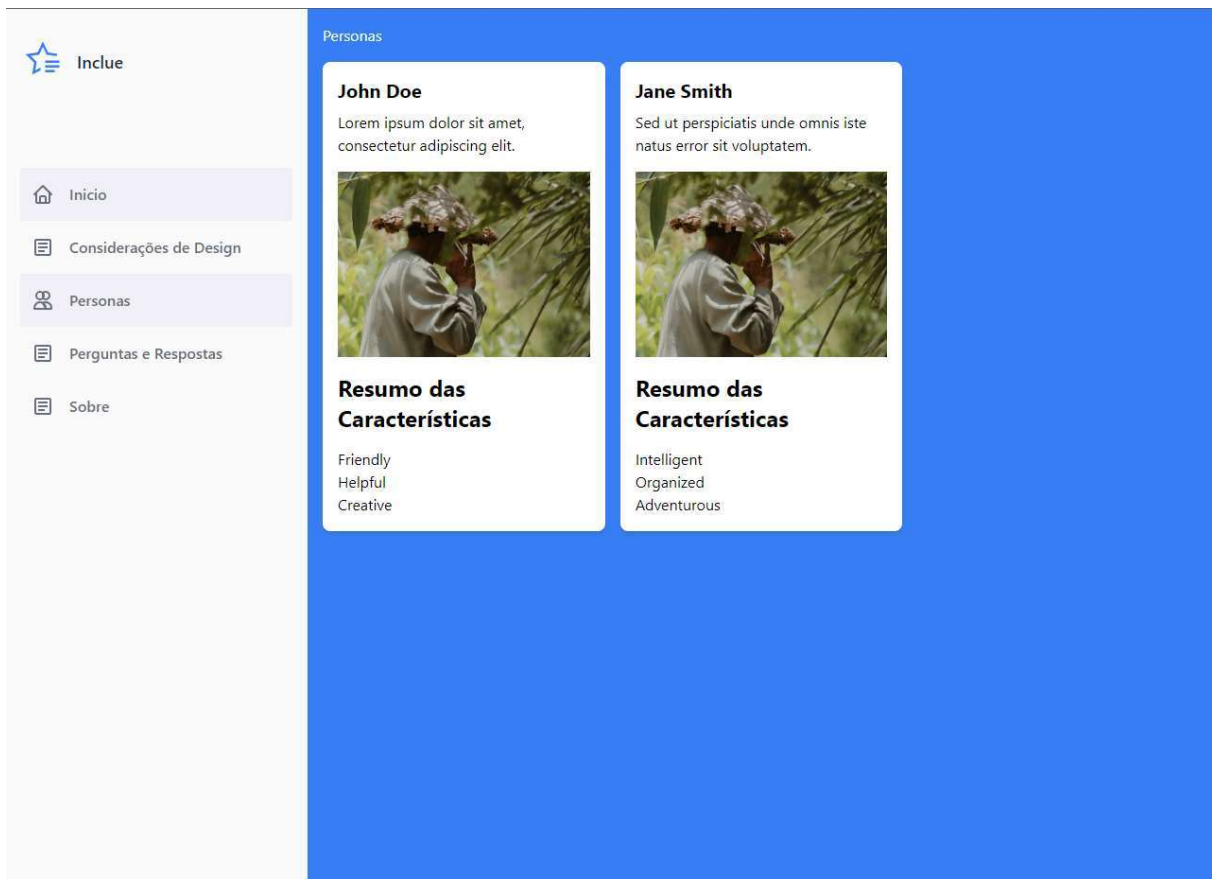
Comando de solicitação

1. importar dummyPersonas.js e implementar utilizando o PersonaCard no return
2. implementar componente Layout
3. Implementar na div o mesmo padrão de card do tailwind

Cenário

Personas

Evidência



Comando 1

```
src > pages > inclue > TS personas.tsx > PersonasPage
1 | import dummyPersonas from './dummyPersonas';
2 | import { PersonaCard } from './components/PersonaCard';
3 |
4 | export default function PersonasPage() {
5 |   return (
6 |     <div>
7 |       {dummyPersonas.map((persona) => (
8 |         <PersonaCard key={persona.id} persona={persona} />
9 |       ))}
10 |     </div>
11 |   );
12 | }
```

Comando 2

```

1  import PersonaCard from '@components/basics/PersonaCard';
2  import { dummyPersonas } from '../db/dummyPersonas';
3
4  export default function PersonasPage() {
5    return (
6      import Layout from '@components/Layout';
7
8      export default function PersonasPage() {
9        return (
10         <Layout>
11           <div>
12             {dummyPersonas.map((persona) => (
13               <PersonaCard key={persona.id}
14                 id={persona.id}
15                 name={persona.name}
16                 description={persona.description}
17                 image={persona.image}
18                 characteristics={persona.characteristics} />
19             ))}
20           </div>
21         </Layout>
22       );
23     );
24     {dummyPersonas.map((persona) => (
25       <PersonaCard key={persona.id}
26         id={persona.id}
27         name={persona.name}
28         description={persona.description}
29         image={persona.image}
30         characteristics={persona.characteristics} />
31     ))}
32   </div>
33 );
34 }

```

Comando 3

```

src > pages > include > TS personastx > PersonasPage
1  import PersonaCard from '@components/basics/PersonaCard';
2  import { dummyPersonas } from '../db/dummyPersonas';
3  import Layout from '@components/Layout';
4
5  export default function PersonasPage() {
6    return (
7      <Layout title="Personas">
8        <div className="grid grid-cols-3 gap-4">
9          {dummyPersonas.map((persona) => (
10            <div key={persona.id} className="bg-white rounded-lg shadow-md p-4">
11              <PersonaCard
12                id={persona.id}
13                name={persona.name}
14                description={persona.description}
15                image={persona.image}
16                characteristics={persona.characteristics}
17              />
18            </div>
19          ))}
20        </div>
21      </Layout>
22    );
23  }

```

Observação

Ao usar o comando 1 ele gerar o código da div utilizando fazendo a iteração do map utilizando o PersonaCard porém os imports estavam errados, precisei fazer a correção de ambos.

```
src > pages > include > TS personas.tsx > PersonasPage > dummyPersonas.map() callback
1 import PersonaCard from '@components/basics/PersonaCard';
2 import { dummyPersonas } from '../../db/dummyPersonas';
3
```

Ao utilizar o comando 2 precisei refatorar a tela manualmente para importar o layout e descrever o título da página conforme a imagem abaixo.

```
src > pages > include > TS personas.tsx > PersonasPage
1 import PersonaCard from '@components/basics/PersonaCard';
2 import { dummyPersonas } from '../../db/dummyPersonas';
3 import Layout from '@components/Layout';
4
5 export default function PersonasPage() {
6   return (
7     <Layout title="Personas">
8       <div>
9         {dummyPersonas.map((persona) => (
10           <PersonaCard key={persona.id}
11             id={persona.id}
12             name={persona.name}
13             description={persona.description}
14             image={persona.image}
15             characteristics={persona.characteristics} />
16         ))}
17       </div>
18     </Layout>
19   );
20 }
21
```

Ao executar o comando 3 ele gerou o layout da tela com o mesmo padrão das demais no formato de card com fundo branco e com a imagem.

1.7 Componente para equipe e grupo

Data de Implementação

13/12/2023

Comando de solicitação

1. gerar div com a propriedades no padrão de card com tailwind

Cenário

Sobre

Evidência

Comando 1

```
src > components > basics > TS AboutCard.tsx > ...
1 | import React from 'react';
2 |
3 | interface AboutCardProps {
4 |   id: number;
5 |   name: string;
6 |   description: string;
7 |   email: string;
8 |   image: string;
9 | }
10
11 export default function AboutCard(props: AboutCardProps) {
12   return (
13     <div className="bg-white rounded-lg shadow-lg p-4">
14       <img src={props.image} alt={props.name} className="w-24 h-24 rounded-full mx-auto mb-4" />
15       <h2 className="text-xl font-bold mb-2">{props.name}</h2>
16       <p className="text-gray-600">{props.description}</p>
17       <p className="text-blue-500">{props.email}</p>
18     </div>
19   );
20 }
```

Observação

Após executar o comando 1 removi o Import do React desnecessário e Refatorei as propriedades conforme as especificações do Figma.

```
src > components > basics > TS AboutCard.tsx > AboutCard
1 | interface AboutCardProps {
2 |   id: number;
3 |   name: string;
4 |   description: string;
5 |   email: string;
6 |   image: string;
7 | }
8 |
9 | export default function AboutCard(props: AboutCardProps) {
10 |   return (
11 |     <div className="bg-white rounded-lg shadow-lg p-4">
12 |       <img src={props.image} alt={props.name} className="w-24 h-24 rounded-full mx-auto mb-4" />
13 |       <h2 className="text-xl font-bold mb-2">{props.name}</h2>
14 |       <p className="text-gray-600">{props.description}</p>
15 |       <p className="text-blue-500">{props.email}</p>
16 |     </div>
17 |   );
18 | }
```

1.8 Tela Sobre

Data de Implementação

13/12/2023

Comando de solicitação

1. importar dummyTeam.js e implementar utilizando o AboutCard no return dentro do componente de Layout
2. preparar uma div para comportar a lista no padrão de card o tailwind
3. refatorar para tornar a div principal e content height: automatico em tailwind

Cenário

Sobre

Evidência

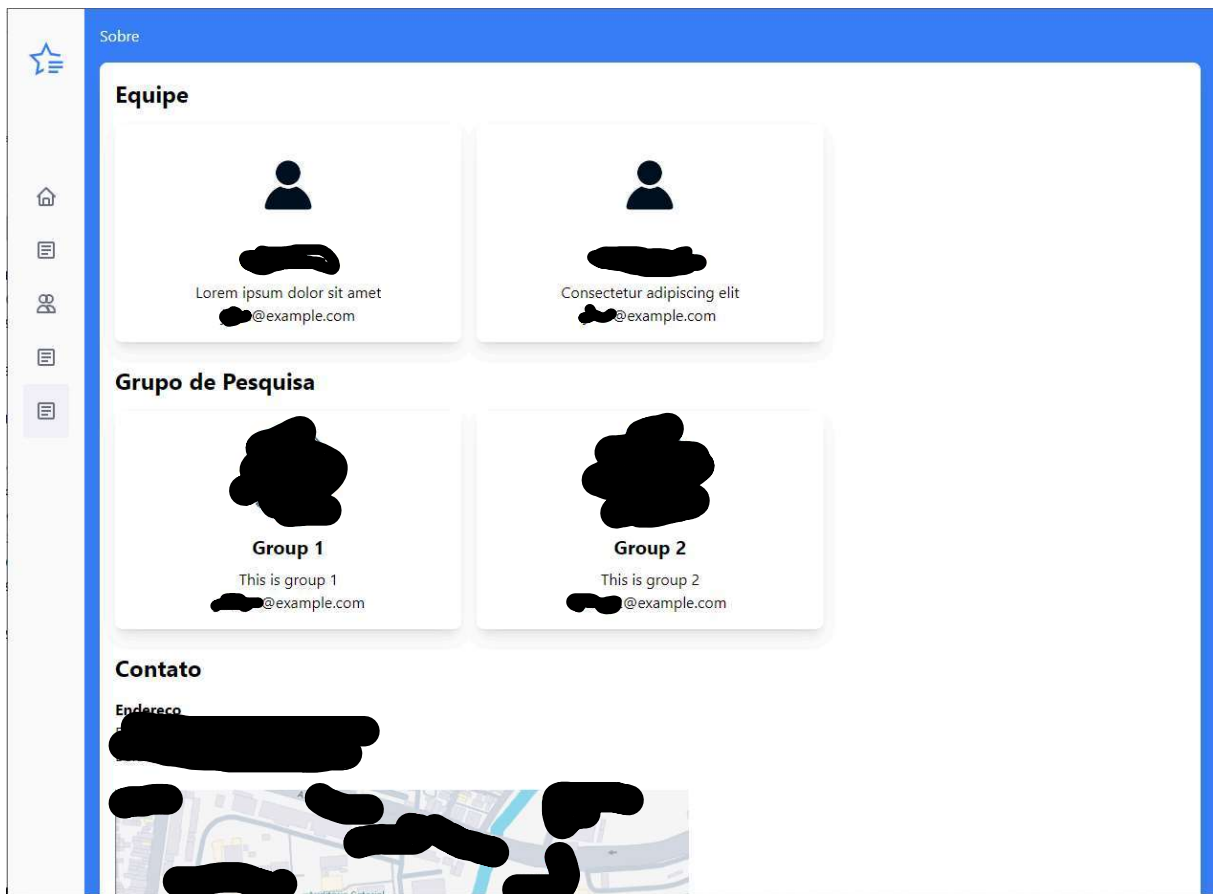
Comando 1

```
src > pages > include > TS about.tsx > AboutPage
1  import dummyTeam from './dummyTeam';
2  import AboutCard from './AboutCard';
3  import Layout from './Layout';
4
5  export default function AboutPage() {
6    return (
7      <Layout>
8        {dummyTeam.map((member) => (
9          <AboutCard key={member.id} member={member} />
10        ))}
11      </Layout>
12    );
13  }
```

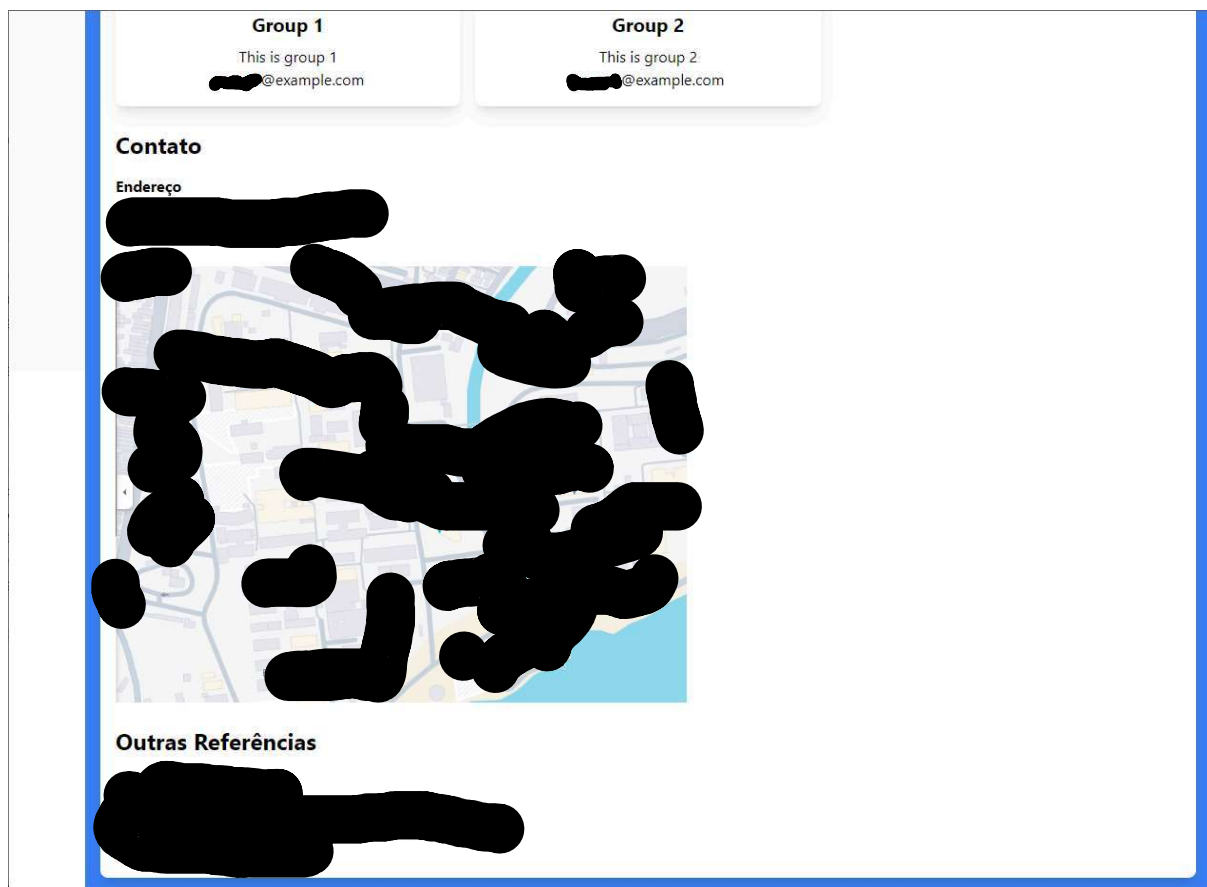
Comando 2

```
src > pages > include > TS abouttsx > AboutPage
1  import AboutCard from '@components/basics/AboutCard';
2  import { dummyTeam } from '../../db/dummyTeam';
3  import Layout from '@components/Layout';
4
5  export default function AboutPage() {
6    return (
7      <Layout title="Sobre">
8        <div className="grid grid-cols-3 gap-4">
9          {dummyTeam.map((member) => (
10             <AboutCard key={member.id}
11               id={member.id}
12               name={member.name}
13               description={member.description}
14               email={member.email}
15               image={member.image}
16             />
17           ))}
18        </div>
19      </Layout>
20    );
21  }
```

Parte 1 da tela implementada



Parte 2 da tela implementada



Observação

Foi necessário refatorar o código no comando 1 para resolver os imports e ajustar o estilo para centralizar os campos do componente.

Também precisei criar as Divs de forma manual para organizar o estilo de uma melhor forma, pois como teria 3 seções na tela o Copilot acabou se perdendo.

Acabei tirando uma cópia de forma manual para utilizar o componente na criação da seção do grupo de pesquisa, contato e outras referências.

src > pages > include > TS abouttsx > AboutPage

```
1 import AboutCard from '@components/basics/AboutCard';
2 import { dummyTeam } from '../../../db/dummyTeam';
3 import { dummyGroup } from '../../../db/dummyGroup';
4 import { dummyAboutReferences } from '../../../db/dummyAboutReferences';
5
6 import Layout from '@components/Layout';
7 import Link from 'next/link';
8
9 export default function AboutPage() {
10   return (
11     <Layout title="Sobre">
12       <div className="bg-white text-justify rounded-lg shadow-lg p-4">
13
14         <div className="text-justify">...
15       </div>
16       <br />
17       <div>
18         <h1 className="text-2xl font-bold mb-4">Grupo de Pesquisa</h1>
19         <div className="grid grid-cols-3 gap-4">
20           {dummyGroup.map((group) => (
21             <AboutCard key={group.id}
22                       id={group.id}
23                       name={group.name}
24                       description={group.description}
25                       email={group.email}
26                       image={group.image}
27             />
28           ))}
29         </div>
30       </div>
31       <br />
32       <div>...
33     </div>
34     <br />
35     <div>...
36   </div>
37 )
38 </Layout>
39 );
40 }
```

src > pages > index > TS about.tsx > AboutPage

```
1 import AboutCard from '@components/basics/AboutCard';
2 import { dummyTeam } from '../../db/dummyTeam';
3 import { dummyGroup } from '../../db/dummyGroup';
4 import { dummyAboutReferences } from '../../db/dummyAboutReferences';
5
6 import Layout from '@components/Layout';
7 import Link from 'next/link';
8
9 export default function AboutPage() {
10   return (
11     <Layout title="Sobre">
12       <div className="bg-white text-justify rounded-lg shadow-lg p-4">
13
14         <div className="text-justify">...
15       </div>
16
17       <br />
18
19       <div>...
20     </div>
21
22     <br />
23
24     <div>
25       <h1 className="text-2xl font-bold mb-4">Contato</h1>
26       <div>
27         <p><strong>Endereço</strong></p>
28         <p>...</p>
29         <p>...</p>
30         
31       </div>
32     </div>
33
34     <br />
35
36     <div>...
37   </div>
38 </Layout>
39 );
40 }
```

```

src > pages > include > TS about.tsx > AboutPage
1  import AboutCard from '@components/basics/AboutCard';
2  import { dummyTeam } from '../../db/dummyTeam';
3  import { dummyGroup } from '../../db/dummyGroup';
4  import { dummyAboutReferences } from '../../db/dummyAboutReferences';
5
6  import Layout from '@components/Layout';
7  import Link from 'next/link';
8
9  export default function AboutPage() {
10     return (
11         <Layout title="Sobre">
12             <div className="bg-white text-justify rounded-lg shadow-lg p-4">
13
14                 <div className="text-justify">...
27             </div>
28             <br />
29             <div>...
42             </div>
43             <br />
44             <div>...
52             </div>
53             <br />
54             <div>
55                 <h1 className="text-2xl font-bold mb-4">Outras Referências</h1>
56                 <ol className="text-justify text-blue-500 underline">
57                     {dummyAboutReferences.map((referece) => {
58                         return (
59                             <li>
60                                 <Link href={referece.url} >
61                                     {referece.description}
62                                 </Link>
63                             </li>
64                         );
65                     })}
66                 </ol>
67             </div>
68         </div>
69     </Layout>
70 );
71 }

```

Inclui de forma automática o className h-auto no layout para deixar a largura da página crescer conforme o as especificações do tailwind, tive dificuldade para conseguir refatorar com o copilot.

src > components > Layout.jsx > Layout

```
1  import Sidebar from "../Sidebar";
2
3
4  const Layout = ({ children, title }) => {
5    return (
6      <div className="h-auto flex flex-row justify-start ">
7        <Sidebar />
8        <div className="bg-primary flex-1 p-4">
9          <div className="mb-4 text-white">
10             <h1>{title}</h1>
11          </div>
12          <div>
13             {children}
14          </div>
15        </div>
16      </div>
17    );
18  };
19
20
21
22  export default Layout;
23
```

```

src > pages > include > TS about.tsx > AboutPage > dummyTeam.map() callback
6   export default function AboutPage() {
7     return (
8       <Layout title="Sobre">
9         <div className="bg-white text-justify rounded-lg shadow-lg p-4">
10
11           <div className="text-justify">
12             <h1 className="text-2xl font-bold mb-4">Equipe</h1>
13             <div className="grid grid-cols-3 gap-4">
14               {dummyTeam.map((member) => (
15                 <AboutCard key={member.id}
16                   id={member.id}
17                   name={member.name}
18                   description={member.description}
19                   email={member.email}
20                   image={member.image}
21                 />
22               ))}
23             </div>
24           </div>
25           <br />
26           <div>
27             <h1 className="text-2xl font-bold mb-4">Grupo de Pesquisa</h1>
28             <div className="grid grid-cols-3 gap-4">
29               {dummyGroup.map((group) => (
30                 <AboutCard key={group.id}
31                   id={group.id}
32                   name={group.name}
33                   description={group.description}
34                   email={group.email}
35                   image={group.image}
36                 />
37               ))}
38             </div>
39           </div>
40           <h1 className="text-2xl font-bold mb-4">Contato</h1>
41           <div className="grid grid-cols-3 gap-4">
42
43           </div>
44         </div>
45       </Layout>
46     );
  
```

1.9 Tela de perguntas e respostas

Data de Implementação

13/12/2023

Comando de solicitação

1. refatorar para incluir um wizard que possua botão previous e next com step initial, steps intermediarios para as questions e step final com botão confirmar
2. incluir input para selecionar um número de 1 a 10 para a quantidade de perguntas
3. criar estrutura para exibir o array dummyQuestion com input check para selecionar a resposta

Cenário

Perguntas e respostas

Evidência

Comando 1

```
src > pages > include > TS questions.tsx > QuestionsPage > handlePrevious
1  import { useState } from 'react';
2  import Layout from '@components/Layout';
3  import { dummyQuestions } from '../../db/dummyQuestions';
4
5  export default function QuestionsPage() {
6    const [step, setStep] = useState(0);
7
8    const handleNext = () => {
9      setStep(step + 1);
10   };
11
12   const handlePrevious = () => {
13     setStep(step - 1);
14   };
15
16   const handleConfirm = () => {
17     // Handle confirm logic here
18   };
19
20   return (
21     <Layout title="Perguntas e Respostas">
22       {step === 0 && (
23         <div>
24           /* Step 0 content */
25           <button onClick={handleNext}>Next</button>
26         </div>
27       )}
28       {step > 0 && step < dummyQuestions.length && (
29         <div>
30           /* Intermediate steps content */
31           <button onClick={handlePrevious}>Previous</button>
32           <button onClick={handleNext}>Next</button>
33         </div>
34       )}
35       {step === dummyQuestions.length && (
36         <div>
37           /* Final step content */
38           <button onClick={handlePrevious}>Previous</button>
39           <button onClick={handleConfirm}>Confirm</button>
40         </div>
41       )}
42     </Layout>
43   );
44 }
```

Comando 2

```

export default function QuestionsPage() {
  const [step, setStep] = useState(0);
  const [questionCount, setQuestionCount] = useState(1); // Add state for question count

  const handleNext = () => {
    setStep(step + 1);
  };

  const handlePrevious = () => {
    setStep(step - 1);
  };

  const handleConfirm = () => {
    // Handle confirm logic here
  };

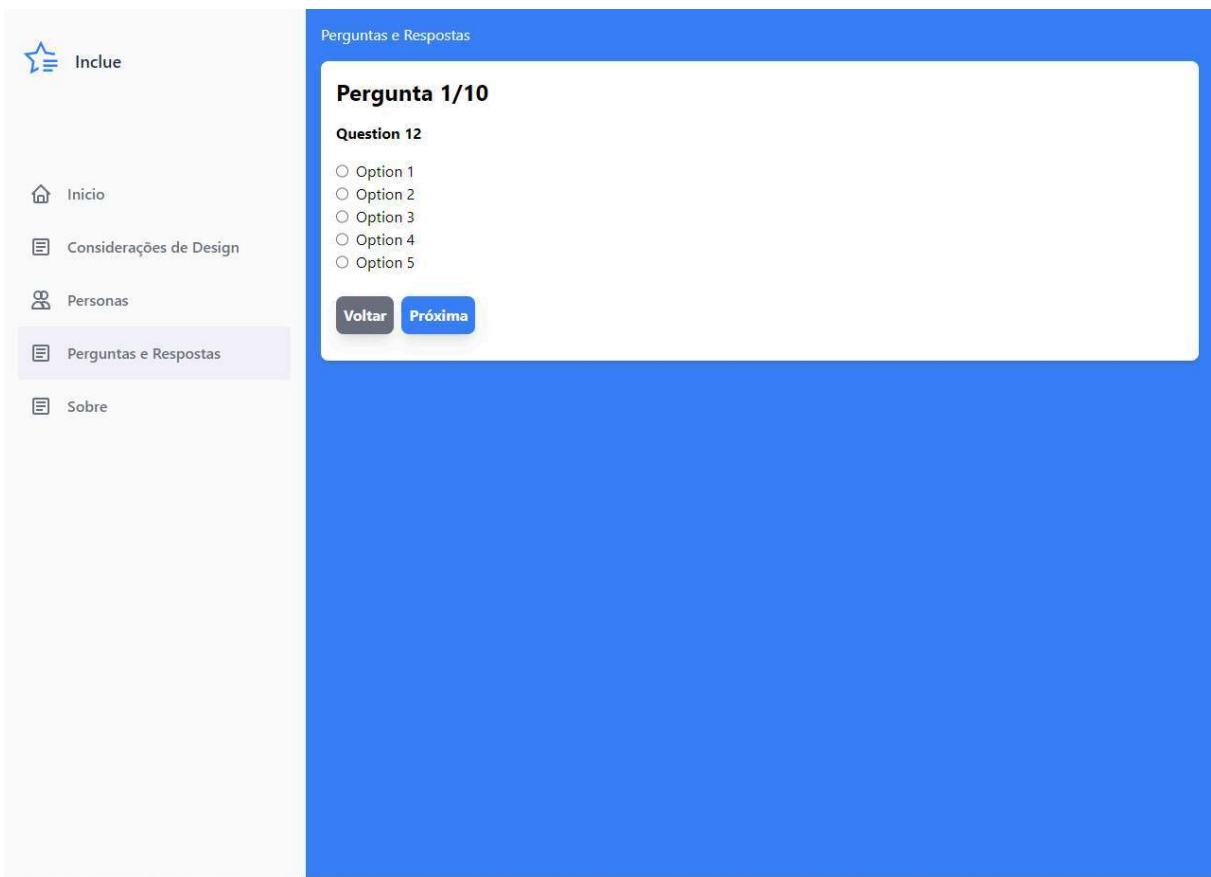
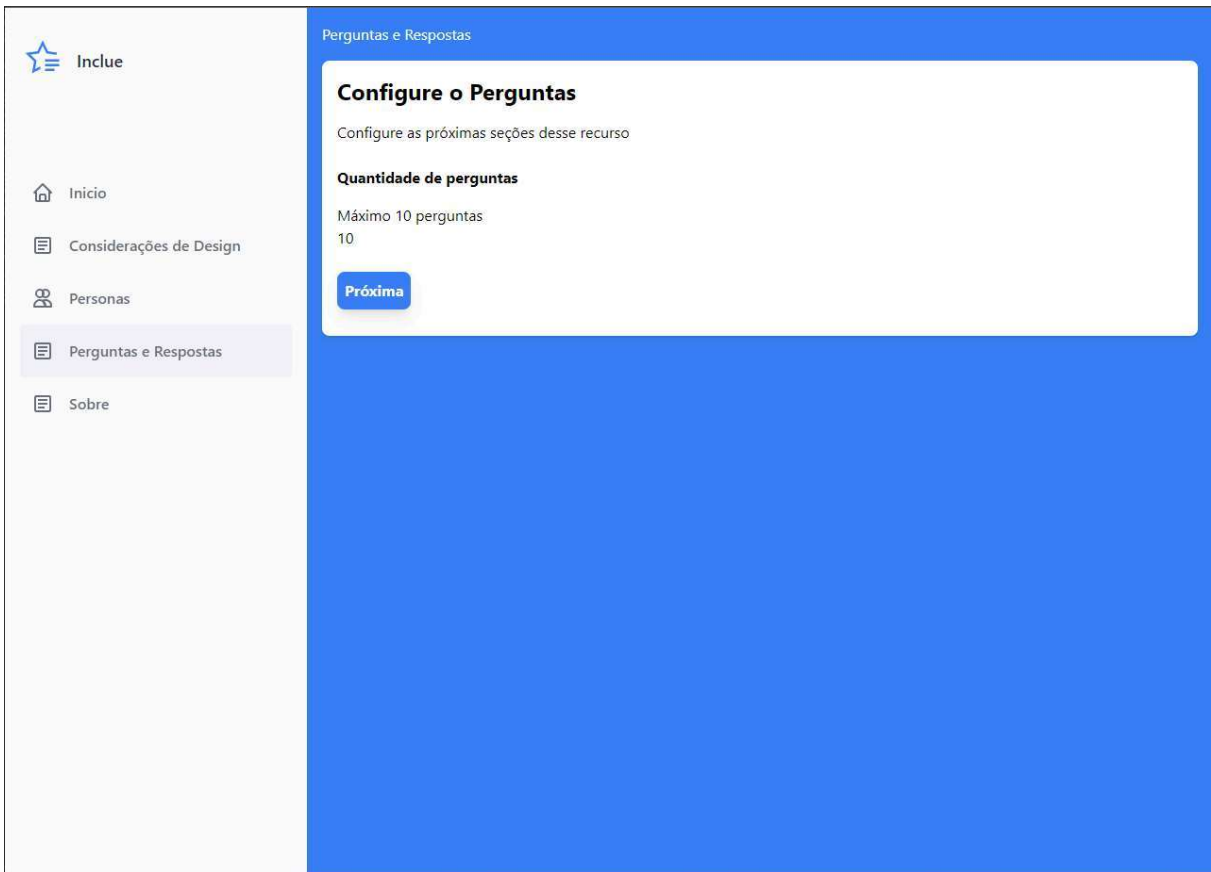
  const handleQuestionCountChange = (event: React.ChangeEvent<HTMLInputElement>) => {
    const count = parseInt(event.target.value);
    setQuestionCount(count);
  };
}

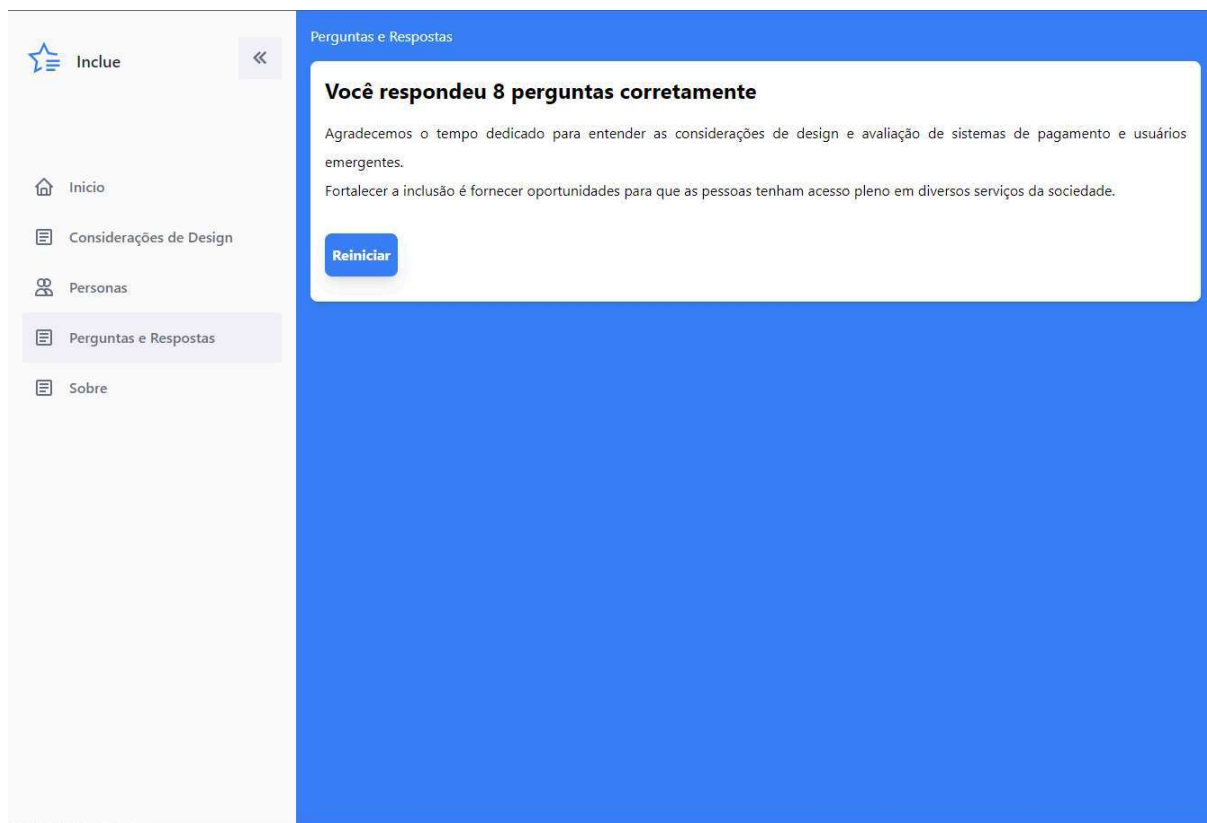
```

```

return (
  <Layout title="Perguntas e Respostas">
    {step === 0 && (
      <div>
        <h1>Configure o Perguntas</h1>
        <p>Configure as próximas seções desse recurso</p>
        <br />
        <h2>Quantidade de perguntas</h2>
        <input
          type="number"
          min="1"
          max="10"
          value={questionCount}
          onChange={handleQuestionCountChange}
        />
        <br />
        <button onClick={handleNext}>Next</button>
      </div>
    )}
  </Layout>
)

```





Observação

Após utilizar os 2 comandos ainda foi necessário refatorar o código para conforme os padrões definidos.

Ao executar o comando 3 o Copilot sugeriu a utilização de um padrão para funcionar a mudança de telas de perguntas em formato de steps, porém foi necessário uma grande refatoração para funcionar.

Foi necessário fazer em grande parte implementação manual, principalmente referente ao estilo e a lógica do wizard para fazer o contador de respostas corretas.

Conclusões

Essa foi a minha primeira experiência desenvolvendo código para web com auxílio de uma ferramenta de IA, de certa forma foi uma excelente experiência e apesar das dificuldades de treinar o que perguntar para o Copilot ele se mostrou uma ferramenta que proporciona um grande auxílio nas dúvidas que temos na hora do desenvolvimento. Tive algumas dificuldades na hora de solicitar um código mais específico e tive que partir para a pesquisa convencional utilizando google ou stackoverflow, mas posso afirmar que a tendência é diminuir essa procura ao utilizar uma ferramenta de IA diretamente no código.

Vejo que essas ferramentas vieram ajudar cada vez os desenvolvedores a tornar seu dia a dia com código mais eficiente, o que falta de fato é a experimentação e saber como solicitar os comandos.

Hoje em dia eu desenvolvo front-end web raramente, vai mais da necessidade da empresa que eu trabalho ou de algum projeto fora do trabalho. A atuação que tenho nos dias atuais é como Techlead de uma equipe que atua com Backend e não tenho que me preocupar com as normas de acessibilidade.

Sobre Copilot esse foi o meu primeiro contato utilizando a ferramenta para gerar código, achei sensacional a forma como ele sugere o auto-completar que ajuda demais no dia a dia para um desenvolvedor. se for para definir a frequência de utilização do Copilot no meu dia a dia, seria não utilizo, devido a precisar utilizar outra ferramenta que é a da empresa que eu Trabalho atualmente chamada StackSpot IA com a base do Codebuddy.

Sobre a implementação basicamente em todas as páginas o modo misto e precisei iniciar o desenvolvimento de forma manual para estruturação do código de todas páginas antes de começar a utilizar o copilot, isso se deu pela falta de experiência com a tecnologia definida, alguns componentes eu copieei de uma estrutura base para iniciar o desenvolvimento do template e nesse momento eu já estava utilizando o autocompletar do Copilot para me auxiliar em algumas refatorações no código copiado.

Na página de Início:

- Iniciei o desenvolvimento de forma manual para estruturação do código;
- Na segunda parte utilizei o prompt para gerar os componentes html e estilo da tela conforme o template definido no figma;
- Por último, foi necessário dar alguns comandos extras para organizar a tela de uma melhor forma.

Na página de Considerações de Design:

- Iniciei o desenvolvimento pelo componente de considerações, esse componente seria responsável por exibir os dados de uma consideração em formato reduzido;
- Na segunda parte implementei a tela de listagem para exibir os dados da listagem de considerações em formato de lista, utilizando o componente estilizado em formato de card para exibir os dados de única consideração;
- Na terceira parte implementei o filtro por categoria com auxílio total do Copilot de forma mista, iniciando com comandos no Prompt e complementando com autocomplete;

- Na quarta parte foi feita a implementação do input de pesquisa por texto, ele foi desenvolvido utilizando comandos específicos no Prompt e pouco modificado no final;
- Na quarta parte complementei o componente de consideração para navegar até a uma tela de detalhe da mesma através de um link;
- Na quinta parte implementei a tela de detalhe da consideração, essa tela irá exibir os dados completos de um único registro, foi necessário refatorar a tela com alguns comando do Copilot para se adequar ao padrão de estilo que já estava sendo utilizado nas demais.
- Por último, foi necessário implementar o botão para voltar a tela de listagem de considerações e dessa forma foi gerado parte do código com comandos no Prompt além de precisar de intervenções manuais no estilo.

Na página de Personas:

- Iniciei o desenvolvimento de forma manual para estruturação do código;
- Na segunda parte utilizei o prompt para gerar os componentes html e estilo da tela conforme o template definido no figma;
- Por último, foi necessário dar alguns comandos extras para organizar a tela de uma melhor forma.

Na página de Sobre:

- Iniciei o desenvolvimento de forma manual para estruturação do código;
- Na segunda parte utilizei o prompt para gerar os componentes html e estilo da tela conforme o template definido no figma;
- Por último, foi necessário dar alguns comandos extras para organizar a tela de uma melhor forma.

Na página de Perguntas e Respostas:

- Iniciei o desenvolvimento de forma manual para estruturação do código;
- Na segunda parte utilizei o prompt para gerar os componentes html e estilo da tela conforme o template definido no figma;
- Por último, foi necessário dar alguns comandos extras para organizar a tela de uma melhor forma.

Sobre geração de código acessível, pude perceber que tiveram alguns momentos que o Copilot sugeriu código com propriedades de acessibilidade, tais como na geração das telas que possuíam imagens, ele gerou a propriedade Alt nessas imagens.

Algumas das vezes que utilizei o recurso para ver outras opções de código, esse código não me atenderam, dessa forma eu precisei mudar a pergunta para o Prompt e ele gerou um código em algumas vezes mais eficiente que o código gerado anteriormente, posso tirar como conclusão disso que é um aprendizado saber como perguntar para se ter um melhor resultado.

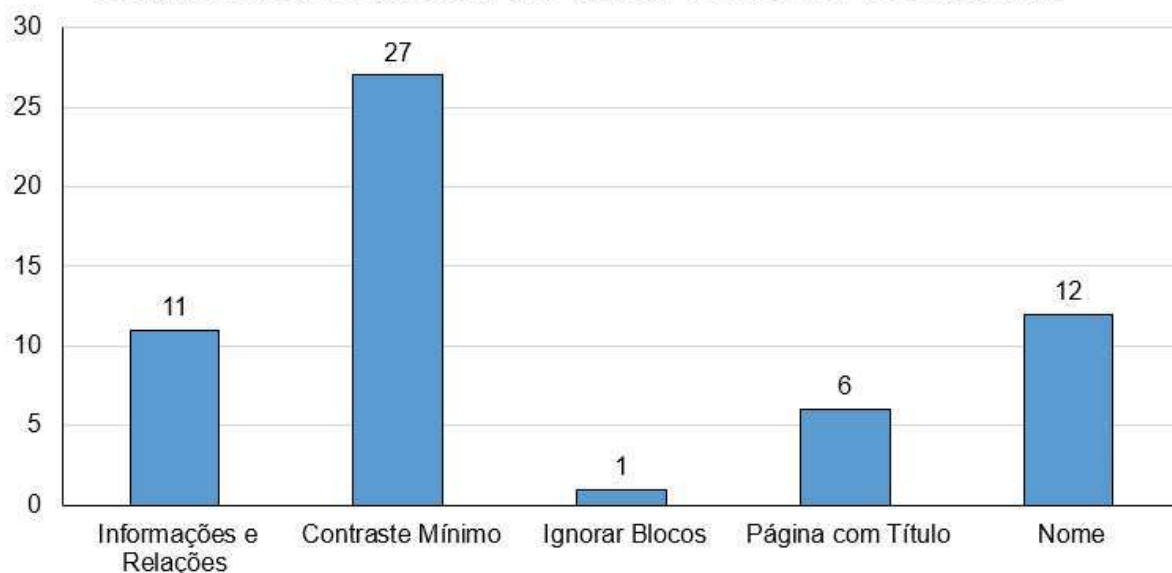
Na grande maioria das vezes o código sugerido pelo prompt do não era um código completo e sempre eu precisei refatorar ou fazer uma implementação por cima da sugestão e especificamente na tela de Perguntas e Resposta eu implementei 90% da lógica de forma manual, porém o Copilot gerou uma estrutura base de steps já funcionando e dessa forma isso evitou que eu fosse buscar código de forma convencional utilizando Google ou Stackoverflow.

Uma sugestão pessoal de melhorias para o Copilot seria, como ele pode me ajudar a identificar e sugerir proativamente os ajustes de acessibilidade para melhorar a experiência de desenvolvimento com front-end web.

Conforme o resultado gerado após a execução do comando para captar a adequação dos critérios de acessibilidade da aplicação utilizando a norma WCAG, identificamos os seguintes problemas:

1. Informações e Relações (1.3.1): 11 problema de estrutura;
2. Contraste Mínimo (1.4.3): 27 problemas de relação de contraste;
3. Ignorar Blocos (2.4.1): 1 problema de bloco de conteúdo;
4. Página com Título (2.4.2): 6 páginas sem a propriedade título;
5. Nome (4.1.2): 12 inputs sem a propriedade nome.

Violações de Acessibilidade por Critério de Sucesso e Pesquisador



2. Refatoração para a Acessibilidade

Essa seção apresenta as interações do programador para a geração do código do projeto com acessibilidade. Desse modo, os códigos foram refatorados a partir de solicitações de geração de código acessível.

Interação 2.1

2.1 Página inicial

Data de Implementação

16/12/2023

Comando de solicitação

1. refatorar para usar head title no lugar do h1

Cenário

Página com Título (2.4.2)

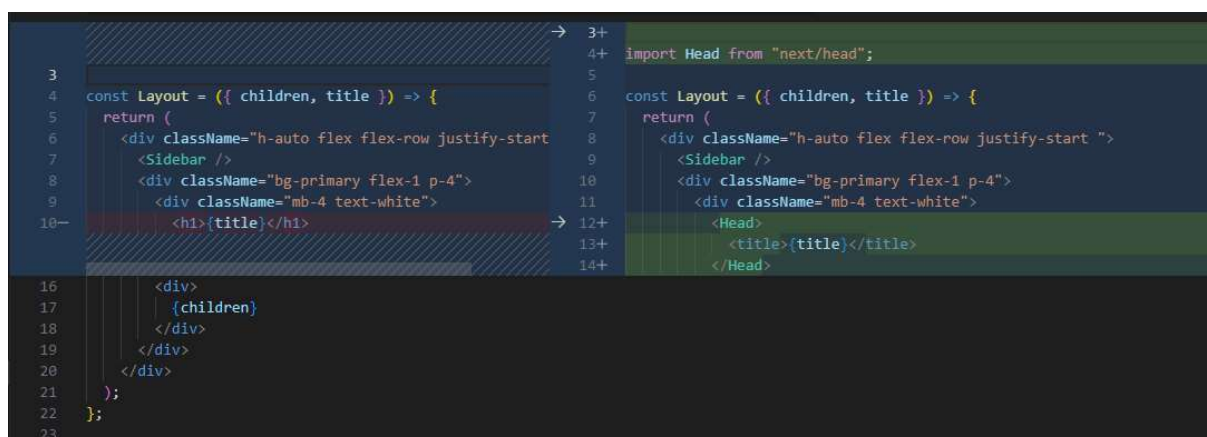
- WCAG2AA.Principle2.Guideline2_4.2_4_2.H25.1.NoTitleEl

Contraste Mínimo (1.4.3)

- WCAG2AA.Principle1.Guideline1_4.1_4_3.G18.Fail

Evidência

Comando 1



```
3
4 const Layout = ({ children, title }) => {
5   return (
6     <div className="h-auto flex flex-row justify-start">
7       <Sidebar />
8       <div className="bg-primary flex-1 p-4">
9         <div className="mb-4 text-white">
10-      <h1>{title}</h1>
16
17       {children}
18     </div>
19   </div>
20 </div>
21 );
22 };
23
```

```
→ 3+
4+ import Head from "next/head";
5
6 const Layout = ({ children, title }) => {
7   return (
8     <div className="h-auto flex flex-row justify-start">
9       <Sidebar />
10      <div className="bg-primary flex-1 p-4">
11        <div className="mb-4 text-white">
12+      <Head>
13+        <title>{title}</title>
14+      </Head>
```

Refatoração manual

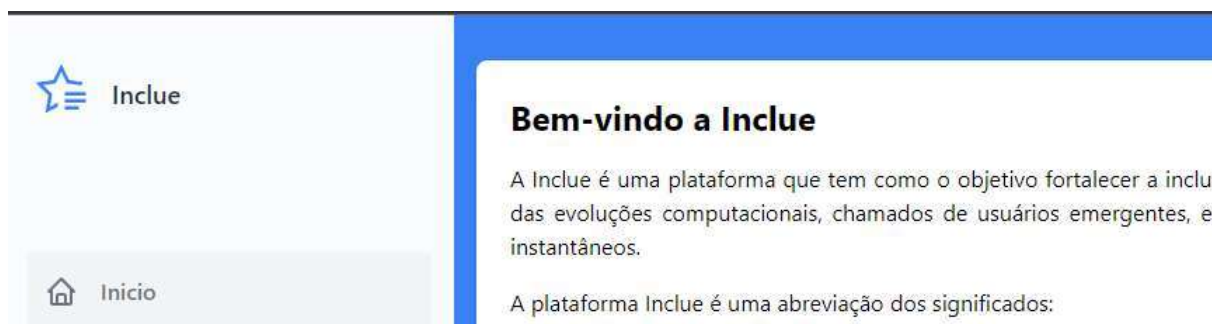
```

src > components > Layout.jsx > Layout
1 | import Head from "next/head";
2 | import Sidebar from "../Sidebar";
3 |
4 |
5 | const Layout = ({ children, title }) => {
6 |   return (
7 |
8 |     <div className="h-auto flex flex-row justify-start ">
9 |       <Head>
10 |        <title>{title}</title>
11 |      </Head>
12 |      <Sidebar />
13 |      <div className="bg-primary flex-1 p-4">
14 |        <div className="mb-4 text-white">
15 |          <h1 className="text-2xl font-bold">{title}</h1>
16 |        </div>
17 |        <div>
18 |          {children}
19 |        </div>
20 |      </div>
21 |    </div>
22 |
23 |   );
24 | };
25 |

```

Observação

Foi necessário refatorar o layout para resolver o erro “Página com Título (2.4.2)” que estava acontecendo para todas as 5 páginas da aplicação, a partir dessa evidência foi executado o comando 1 e ele sugeriu a inclusão das tags title <Head> e <title>. Após executar o comando ele sobrescreve a tag <h1> que era necessária para exibir o título da página de forma visual, ao incluir o <h1> novamente o Copilot auto-completou gerando a tag com um estilo melhorado e ele acabou resolvendo o problema de contraste sem eu perceber.





Obs.: Após resolver esse problema, outras telas que utilizavam o Layout e tinham o mesmo erro foram corrigidas.

2.2 Página Inicial

Data de Implementação

16/12/2023

Comando de solicitação

1. refatorar tela para atender a norma WCAG2AA Principle1 Guideline1_4 1_4_3 G18
Fail
2. refatorar a div para atender a norma WCAG2AA 1_4_3 G18

Cenário

Contraste Mínimo (1.4.3)

- WCAG2AA.Principle1.Guideline1_4.1_4_3.G18.Fail

Evidência

```

1 import Layout from "../components/Layout";
2
3 export default function Home() {
4   return (
5     <Layout title="Início">
6       <div className="bg-white rounded-lg shadow-lg p-6" role="region" aria-labelledby="welcome-heading" aria-label="Bem-vindo a Incluir">
7         <div className="text-justify">
8           <h1 id="welcome-heading" className="text-2xl font-bold mb-4">Bem-vindo a Incluir</h1>
9         </div>
10        <div className="text-justify">
11          <p className="mb-4">A Incluir é uma plataforma que tem como o objetivo fortalecer a inclusão digital de pessoas que historicamente estiveram d
12          <p className="mb-4">A plataforma Incluir é uma abreviação dos significados:</p>
13          <ul className="list-disc ml-8 mb-4">
14            <li><strong>Incluir:</strong> incluir os usuários emergentes aos sistemas interativos</li>
15            <li><strong>Nortear:</strong> nortear os designers sobre as características de interação de usuário</li>
16            <li><strong>Conectar:</strong> conectar os Sistemas de Pagamento Instantâneos como uma alternativa de gestão financeira aos usuários emerge
17            <li><strong>Legitimar:</strong> legitimar os usuários emergentes como protagonistas dos serviços digitais e desenvolvimento sustentável</li>
18          </ul>
19          <p className="mb-4">Desse modo, nessa plataforma você interagirá com:</p>
20          <ul className="list-disc ml-8 mb-4">
21            <li><strong>Considerações de design</strong> e avaliação para sistemas de pagamento instantâneos</li>
22            <li><strong>Personas</strong> que apresentam as características sociais, digitais e financeiras de usuários emergentes</li>
23            <li><strong>Perguntas e Respostas</strong> relacionadas às considerações de design e avaliação para usuários emergentes e sistemas de pagam
24            <li><strong>Sobre</strong> com a apresentação da equipe de desenvolvimento e os documentos de origem dessa plataforma</li>
25          </ul>
26          <p>Desde já, agradecemos seu interesse em conhecer essa proposta de um ambiente digital mais inclusivo.</p>
27        </div>
28      </div>
29    </Layout>
30  )
31 }
32

```

Refatoração manual do Sidebar

```

src > components > Sidebar.jsx > [0] Sidebar > [0] menuItems.map() callback
96
97   <button
98     className={collapseIconClasses}
99     onClick={handleSidebarToggle}>
100     <CollapseIcon />
101   </button>
102 </div>
103
104 <div className="flex flex-col items-start mt-24">
105   {menuItems.map(({ icon: Icon, ...menu }) => {
106     const classes = getNavItemClasses(menu);
107     return (
108       <div className={classes}>
109         <Link href={{pathname: menu.link}} >
110           <div className="flex py-4 px-3 items-center w-full h-full">
111             <div style={{ width: "2.5rem" }}>
112               <Icon />
113             </div>
114             {!toggleCollapse && (
115               <span
116                 className={classNames(
117                   "text-md font-medium",
118                   toggleCollapse ? "text-text-light" : ""
119                 )}
120               {menu.label}
121             </span>
122             )}
123           </div>
124         </Link>
125       </div>
126     );
127   })}
128 </div>
129 </div>
130 </div>
131 );
132 ;

```

```
src > components > Sidebar.jsx > Sidebar > menuItems.map() callback
96     <button
97       className={collapseIconClasses}
98       onClick={handleSidebarToggle}>
99       <CollapseIcon />
100     </button>
101   )}
102 </div>
103
104 <div className="flex flex-col items-start mt-24">
105   {menuItems.map(({ icon: Icon, ...menu }) => {
106     const classes = getNavItemClasses(menu);
107     return (
108       <div className={classes}>
109         <Link href={{pathname: menu.link}} >
110           <div className="flex py-4 px-3 items-center w-full h-full">
111             <div style={{ width: "2.5rem" }}>
112               <Icon />
113             </div>
114             {!!toggleCollapse && (
115               <span
116                 className={classNames(
117                   "text-md font-medium"
118                 )}
119               >
120                 {menu.label}
121               </span>
122             )}
123           </div>
124         </Link>
125       </div>
126     );
127   })}
128 </div>
129 </div>
130 </div>
131 );
132 };
```

Sugestão do auto-complete

```
src > components > Sidebar.jsx > Sidebar > menuItems.map() callback
96     <button
97       className={collapseIconClasses}
98       onClick={handleSidebarToggle}>
99       <CollapseIcon />
100     </button>
101   )}
102 </div>
103
104 <div className="flex flex-col items-start mt-24">
105   {menuItems.map(({ icon: Icon, ...menu }) => {
106     const classes = getNavItemClasses(menu);
107     return (
108       <div className={classes}>
109         <Link href={{pathname: menu.link}} >
110           <div className="flex py-4 px-3 items-center w-full h-full">
111             <div style={{ width: "2.5rem" }}>
112               <Icon />
113             </div>
114             {!toggleCollapse && (
115               <span
116                 className={classNames(
117                   "text-md font-medium",
118                   "text-text ml-4"
119                 )}
120               {menu.label}
121             </span>
122             )}
123           </div>
124         </Link>
125       </div>
126     );
127   })}
128 </div>
129 </div>
130 </div>
131 );
132 };
```

Observação

Utilizei o comando 1 para refatorar a tela e não obtive sucesso, preciso ser mais específico.

Ao utilizar o comando 2 ele sugeriu um código novo que também não teve sucesso para corrigir o problema.

Foi identificado que o problema não era na página codificada e sim no Sidebar do template que nós copiamos da internet para acelerar o desenvolvimento, dessa forma tentei fazer a correção utilizando também o comando 2 na side bar, porém ele não corrigiu o problema. O problema foi corrigido após eu remover da propriedade “className” o “text-text-light” e o próprio Copilot fez uma sugestão no autocompletar para deixar uma cor mais acessível.

Obs.: Após resolver esse problema, outras telas que utilizavam o Sidebar e tinham os mesmos erros foram corrigidas.

2.3 Página da listagem de considerações de design

Data de Implementação

16/12/2023

Comando de solicitação

1. resolve accessibility problem for WCAG2AA 4.1.2 H91 InputText Name

Cenário

Nome (4.1.2)

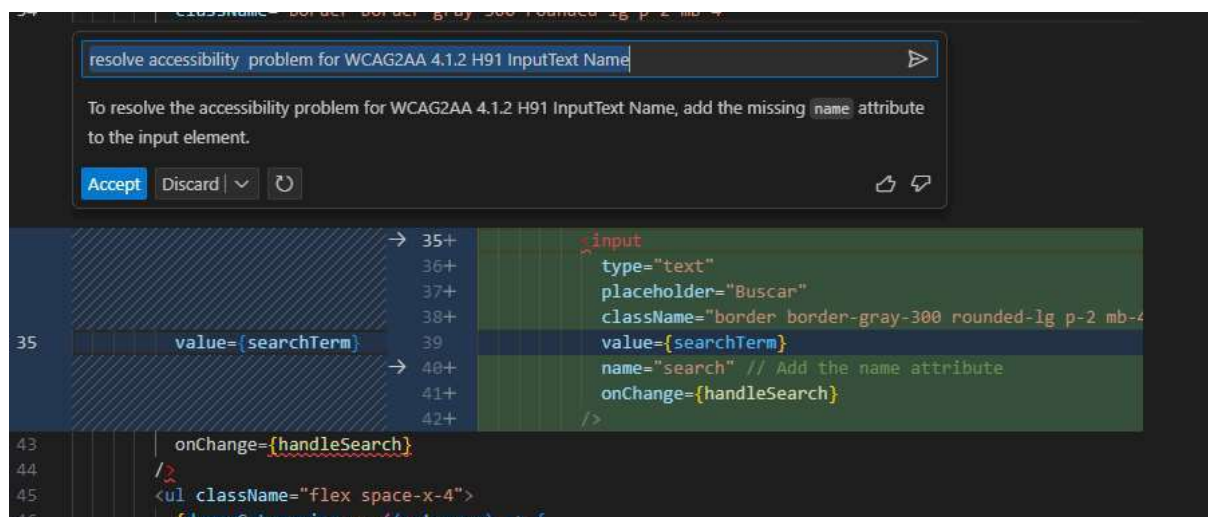
- WCAG2AA.Principle4.Guideline4_1.4_1_2.H91.InputText.Name

Informações e Relações (1.3.1)

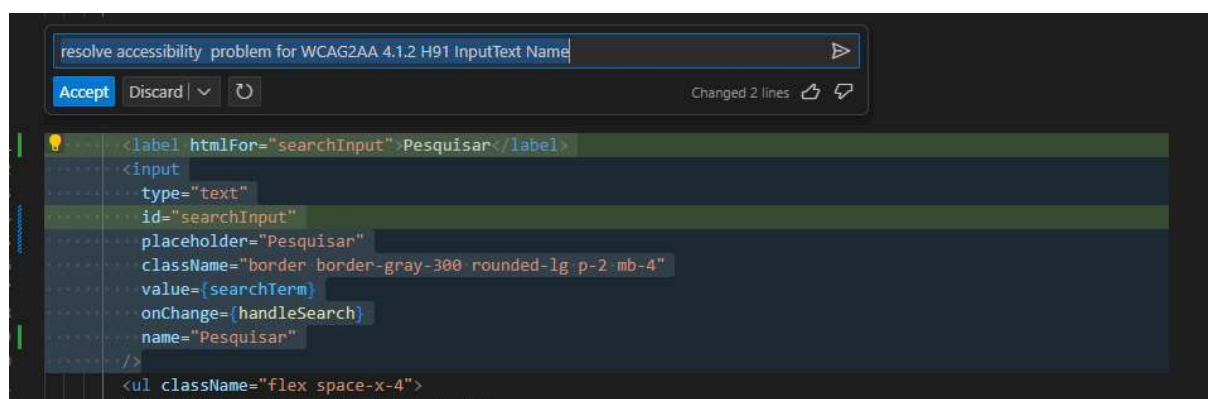
- WCAG2AA.Principle1.Guideline1_3.1_3_1.F68

Evidência

Comando 1 teste 1



Comando 1 teste 2



Observação

Escrever sobre o retorno do prompt e que sugeriu como comentario tbm

No primeiro teste executei o comando 1 e ele gerou o name para o input, porém ao executar novamente o teste de acessibilidade ele acusou que o erro permanecia.

Executei novamente o comando 1 selecionando o Input desde a linha 38 até a linha 31 conforme a imagem abaixo, o Copilot incluiu as propriedades “name” e “placeholder” e uma “label” para o meu “input”.

```
30 </div>
31 <input
32   type="text"
33   placeholder="Pesquisar"
34   className="border border-gray-300 rounded-lg p-2 mb-4"
35   value={searchTerm}
36   onChange={handleSearch}
37   name="Pesquisar"
38 />
39 <ul className="flex space-x-4">
```

○

2.4 Página da listagem de considerações de design

Data de Implementação

16/12/2023

Comando de solicitação

1. resolve accessibility problem for WCAG2AA 1.4.3 G18 contrast ratio
2. replicar a solução do span da linha 48

Cenário

Contraste Mínimo (1.4.3)

- WCAG2AA.Principle1.Guideline1_4.1_4_3.G18.Fail

Evidência

Primeira execução do comando 1

```
5 return (
6   <li className="bg-primary text-justify text-white rounded-lg">
7     <Link href={`/include/considerations?filter=${category}`} >
8       {category} → 48+ <span className="text-white">{category}</span>
9     </Link>
10  </li>
```

Segunda execução do comando 1

```
return [  
  <li className="bg-primary text-justify text-white rounded-lg">  
    <Link href={`/incluir/considerations?filter=${category}`} >  
      <a className="bg-blue-600 text-white rounded-lg" style={{ backgroundColor: "#2b71e6" }}>  
        {category}  
      </a>  
    </Link>  
  </li>  
</li>
```

Resolução manual após a segunda execução

```
return [  
  <li className="bg-primary text-justify text-white rounded-lg">  
    <Link href={`/incluir/considerations?filter=${category}`} >  
      <span className="bg-blue-600 text-white rounded-lg" style={{ backgroundColor: "#2b71e6" }}>  
        {category}  
      </span>  
    </Link>  
  </li>  
</li>
```

Comando 2

```
43 <ul className="flex space-x-4">  
44   {dummyCategories.map((category) => {  
45     return (  
46       <li className="bg-primary text-justify text-white rounded-lg">  
47         <Link href={`/incluir/considerations?filter=${category}`} >  
48           <span className="bg-blue-600 text-white rounded-lg" style={{ backgroundColor: "#2b71e6" }}>  
49             {category}  
50           </span>  
51         </Link>  
52       </li>  
53     );  
54   })}  
55 </ul>
```

replicar a solução do span da linha 48

Accept Discard | v ↺ Changed 3 lines

55	<li className="bg-primary text-justify text-white rounded-lg">	→ 57+	
56	<Link href={`/incluir/considerations`} >	→ 58	Todos
57	Todos	→ 59+	
60	</Link>		
61			
62			

Resolução manual do componente de considerações

```

src > components > basics > TS ConsiderationCard.tsx > ConsiderationCard
1  import Link from "next/link"
2
3  interface ConsiderationCardProps {
4    id: number
5    title: string
6    category: string
7    content: string
8  }
9
10 export default function ConsiderationCard(props: ConsiderationCardProps) {
11   return (
12     <div className="flex flex-col justify-center items-center">
13       <div className="font-bold text-xl mb-2">
14         {props.title}
15       </div>
16       <div className="bg-primary text-justify text-white rounded-lg">
17         <span className="bg-blue-600 text-white rounded-lg" style={{ backgroundColor: "#2b71e6" }}>
18           {props.category}
19         </span>
20       </div>
21       <div className="text-justify">
22         {props.content.substring(0, 100) + "..."}
23       </div>
24       <div className="text-blue-500 underline text-end">
25         <Link href={` /incluir/consideration?id=${props.id}`} >
26           Saiba mais
27         </Link>
28       </div>
29     </div>
30   )
31 }

```

Observação

Ao executar o comando 1 ele não resolveu o problema de acessibilidade relatado.

Ao executar o comando novamente ele sugeriu incluir a tag `<a>`, se eu aceitar ela tem bug no React que impossibilita a utilização da tag dentro de um Link e por esse motivo precisei remover a tag `<a>` e substituir por um span.

Executei o comando 2 para copiar a solução de `<span>` para o link da categoria todos.

Por último identifiquei o problema no componente das considerações, tentei executar o comando 2 e 3 para replicar a solução mas o que resolveu o problema foi fazer uma cópia manual do `<span>` com o estilo necessário.

2.5 Página de detalhe das considerações de design

Data de Implementação

16/12/2023

Comando de solicitação

Nenhum

Cenário

Contraste Mínimo (1.4.3)

- WCAG2AA.Principle1.Guideline1_4.1_4_3.G18.Fail

Evidência

```
1  import Link from "next/link"
2
3  interface ConsiderationCardProps {
4    id: number
5    title: string
6    category: string
7    content: string
8  }
9
10 export default function ConsiderationCard(props: ConsiderationCardProps) {
11   return (
12     <div className="flex flex-col justify-center items-center">
13       <div className="font-bold text-xl mb-2">
14         {props.title}
15       </div>
16       <div className="bg-primary text-justify text-white rounded-lg">
17         <span className="bg-blue-600 text-white rounded-lg" style={{ backgroundColor: "#2b71e5", color: "white" }}>
18           {props.category}
19         </span>
20       </div>
21       <div className="text-justify">
22         {props.content.substring(0, 100) + "..."}
23       </div>
24       <div className="text-blue-500 underline text-end">
25         <Link href={` /incluir/consideration?id=${props.id}`} >
26           Saiba mais
27         </Link>
28       </div>
29     </div>
30   )
31 }
```

Observação

Copiei a solução indicada pelo Copilot no item 2.4 através, utilizando um span com o estilo que foi definido.

2.6 Tela Sobre

Data de Implementação

16/12/2023

Comando de solicitação

1. resolve Iframe element requires a non-empty title attribute that identifies the frame

Cenário

Ignorar Blocos (2.4.1)

- WCAG2AA.Principle2.Guideline2_4.2_4_1.H64.1

Evidência

```
17      {dummyTeam.map((member) => {
18          <AboutCard key={member.id}
19                  id={member.id}
20                  name={member.name}
21                  description={member.description}
22                  email={member.email}
23                  image={member.image}
24                  />
25      })}
26  </div>
27 </div>
28 <br />
29 > <div> ...
42 </div>
43 <br />
44 <div>
45     <h1 className="text-2xl font-bold mb-4">Contato</h1>
46     <div>
47         <p><strong>Endereço</strong></p>
48         <p>Rua Augusto Corrêa - nº 01 ☐ Guamã</p>
49         <p>Belém, Pará, Brasil ☐ CEP: 66075110</p>
50         <br />
51         <iframe title="Google Maps"
52                 style={{width:"100%", height:"400px"}} loading="lazy"
53                 src="https://www.google.com/maps/embed?pb=!1m18!1m12!1m3!1d3988.497609398409!2d
54             </div>
55     </div>
56 <br />
57 <div>
58     <h1 className="text-2xl font-bold mb-4">Outras Referências</h1>
59     <ol className="text-justify text-blue-500 underline">
60         {dummyAboutReferences.map((reference) => {
61             return (
62                 <li>
63                     <Link href={reference.url} >
64                         {reference.description}
65                     </Link>
66                 </li>
```

Observação

A execução do comando 1 resolveu o problema incluindo a propriedade “title”.

2.7 Tela de perguntas e respostas

Data de Implementação

16/12/2023

Comando de solicitação

1. to resolve This numberinput element does not have a name available to an accessibility API.
2. incluir label para o input

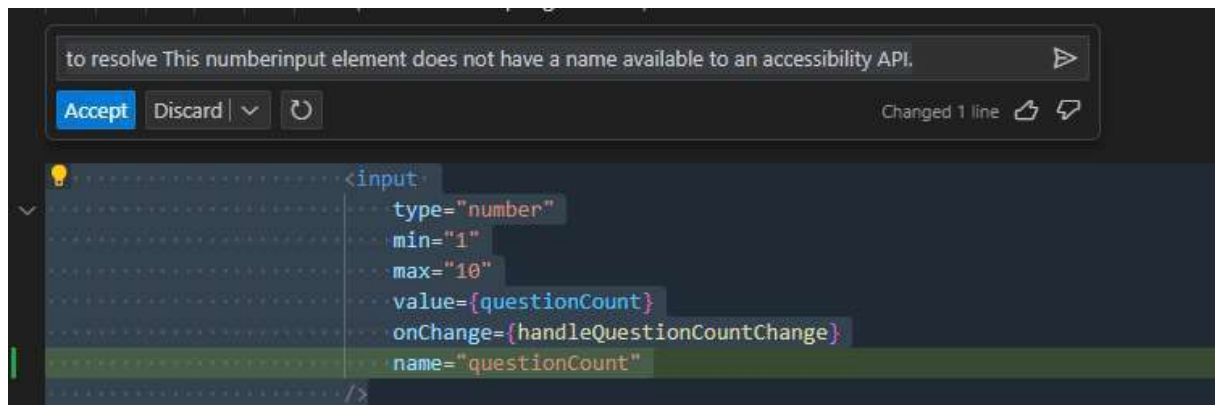
Cenário

Nome (4.1.2)

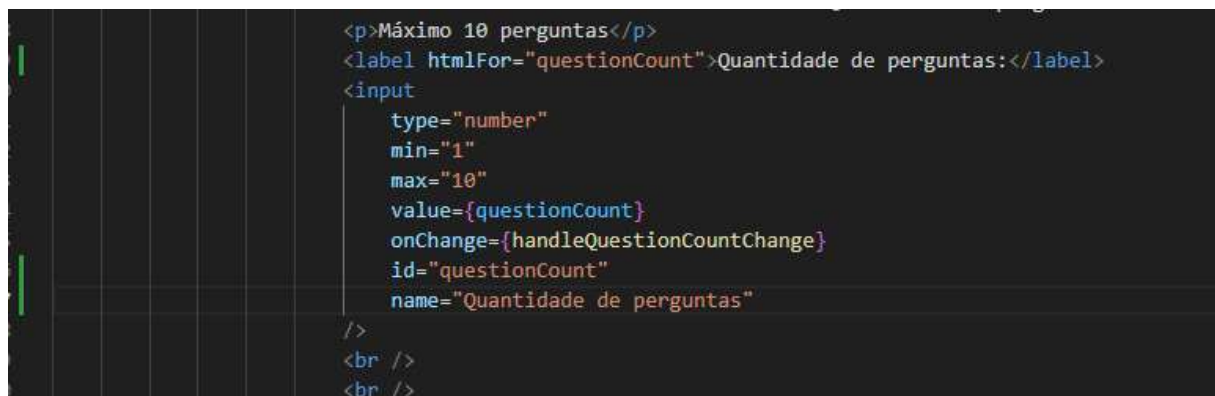
- WCAG2AA.Principle4.Guideline4_1.4_1_2.H91.InputNumber.Name

Evidência

Comando 1



Comando 2



Inclue

Início

Considerações de Design

Personas

Perguntas e Respostas

Sobre

Perguntas e Respostas

Configure o Perguntas

Configure as próximas seções desse recurso

Quantidade de perguntas

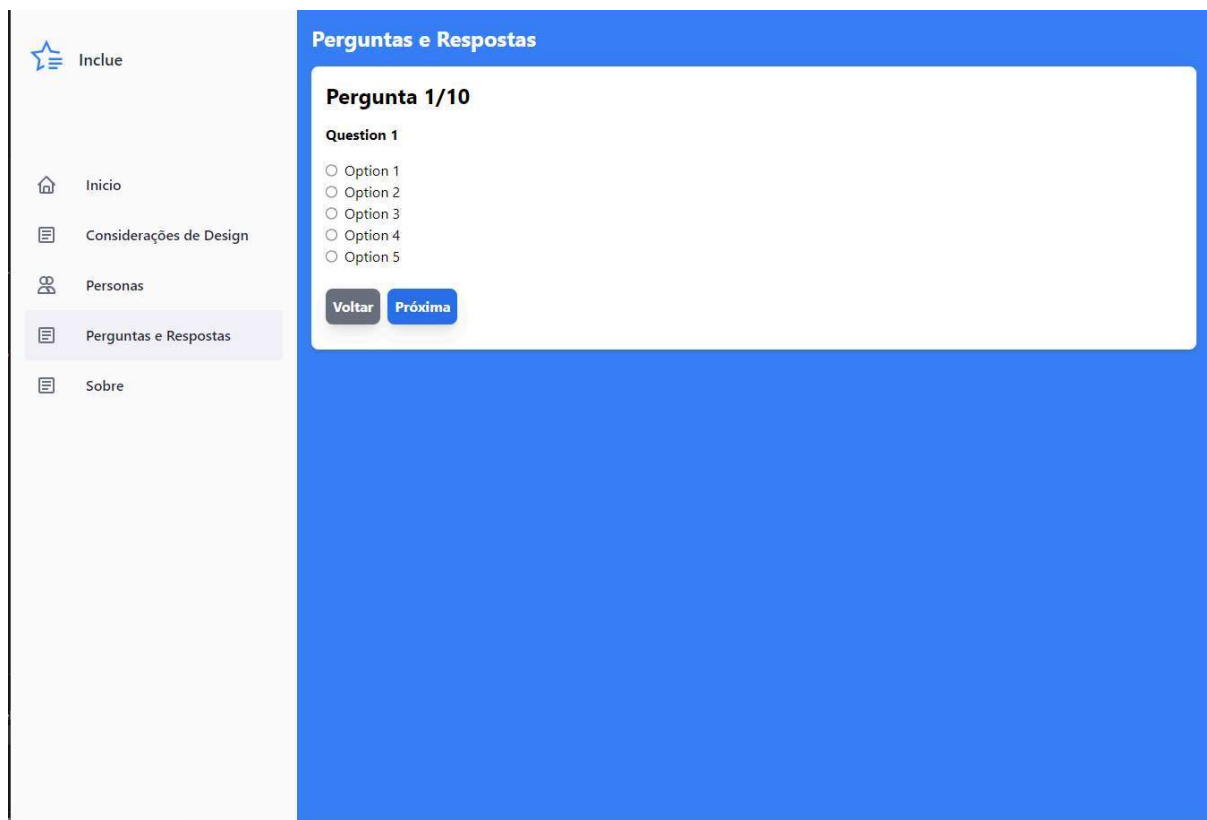
Máximo 10 perguntas

Quantidade de perguntas

Próxima

Intervenção manual no input checkbox de opções

```
<h1 className="text-1xl font-bold mb-4">{actualQuestion.title}</h1>
{actualQuestion.options.map((option) => (
  <div className='flex gap-2'>
    <input
      type="radio"
      id={actualQuestion.id+''+option.id}
      value={option.id}
      name={option.text}
      checked={optionAuxiliar === option.id}
      onChange={(e) => handleOptionSelection(parseInt(e.target.value))}
    <label htmlFor={actualQuestion.id+''+option.id}>{option.text}</label>
  </div>
)}
<br />
```



Observação

O comando 1 resolveu os 2 problemas e só precisei fazer algumas alterações.

Os erros de “input” para “checkbox” foram feitas de forma manual, fiz inclusão do “label” e o “name” para os 2 contextos de checkbox definindo um “id” dinâmico para cada input da lista.

2.8 Tela de perguntas e respostas

Data de Implementação

16/12/2023

Comando de solicitação

1. to resolve problem for insufficient contrast, change background to #2b71e6 in tailwind

Cenário

Contraste Mínimo (1.4.3)

- WCAG2AA.Principle1.Guideline1_4.1_4_3.G18.Fail

Evidência

Comando 1

```
    <div className='flex gap-2'>
      <button className="bg-blue-600 flex rounded-lg
        shadow-lg p-2 mb-3 gap-2 text-white
        items-center font-bold"
        onClick={handleNext}>Próxima</button>
    </div>
  </div>
```

Intervenção manual botões

```
1    <div className='flex gap-2'>
2      <button className="flex rounded-lg shadow-lg p-2 mb-3 gap-2 items-center"
3        <span className="bg-blue-600 text-white rounded-lg">
4          Voltar
5        </span>
6      </button>
7      <button className="flex rounded-lg shadow-lg p-2 mb-3 gap-2 items-center font-b
8        style={{ backgroundColor: "#2b71e6" }} onClick={handleNext}>
9        <span className="bg-blue-600 text-white rounded-lg"
10          style={{ backgroundColor: "#2b71e6" }} >
11          Próxima
12        </span>
13      </button>
14    </div>
```

```
171    ))
172    <br />
173    <div className='flex gap-2'>
174      <button className="flex rounded-lg shadow-lg p-2 mb-3 gap-2 items-center bg-
175        <span className="bg-blue-600 text-white rounded-lg">
176          Voltar
177        </span>
178      </button>
179      <button className="flex rounded-lg shadow-lg p-2 mb-3 gap-2 items-center f
180        style={{ backgroundColor: "#2b71e6" }} onClick={handleConfirm}>
181        <span className="bg-blue-600 text-white rounded-lg"
182          style={{ backgroundColor: "#2b71e6" }}>
183          Finalizar
184        </span>
185      </button>
186    </div>
187  </div>
188  ))
```

```

91
92     {step > selectedQuestions.length && [
93       <div className='text-justify leading-loose'>
94         <h1 className="text-2xl font-bold mb-4">Você respondeu {correctQuestionsCount}.
95         <p>Agradecemos o tempo dedicado para entender as considerações de design e ava
96         <p>Fortalecer a inclusão é fornecer oportunidades para que as pessoas tenham a
97         <br />
98         <div className='flex gap-2'>
99           <button className="flex rounded-lg shadow-lg p-2 mb-3 gap-2 items-center fo
100             style={{ backgroundColor: "#2b71e6" }} onClick={handleReset}>
101             <span className="bg-blue-600 text-white rounded-lg"
102             style={{ backgroundColor: "#2b71e6" }}>
103               Reiniciar
104             </span>
105           </button>
106         </div>
107       </div>
108     ]
109   </div>
110 </layout>

```

Observação

Ao executar o comando 1 ele não resolveu o problema e ainda quebrou a cor, precisei atuar de forma manual para resolver o problema de botões com o azul fora do padrão de contraste das normas de acessibilidade.

Perguntas e Respostas

Configure o Perguntas

Configure as próximas seções desse recurso

Quantidade de perguntas

Máximo 10 perguntas

Quantidade de perguntas:10

Conclusões

Essa interação foi mais simples que a primeira devido a somente precisarmos refatorar as telas com base no relatório gerado pela ferramenta “pally” e dos 57 erros identificados todos foram resolvidos.

O que pude perceber sobre a utilização do Copilot:

- A dificuldade de informar um comando que atendesse o critério para refatorar o código que funcionasse;
- A dificuldade do Copilot para sugerir uma resolução mais assertiva do ponto de vista de Design Patterns e componentização do projeto, ainda é uma responsabilidade do desenvolvedor Front-end web;
- Em alguns momentos eu consegui replicar o código gerado em uma determinada linha do mesmo arquivo para outra, por exemplo quando tentei refatorar o estilo de uma “<div>” para outra, porém ao tentar pegar de outro arquivo ele teve dificuldades e precisei fazer de forma manual;
- Apesar do Copilot conseguir identificar o problema dos “inputs” sem o “name” e sem o “label” o não determinismo, o não determinismo foi um fator que acabou atrapalhando na hora de gerar essa refatoração pois tive que tentar executar o comando no prompt mais vezes que o habitual para acertar uma pergunta que ele fizesse a inclusão correta dessa propriedades;
- Tivemos também um problema com a ferramenta “pally” na tela de perguntas e respostas, ela não conseguiu gerar o relatório para todos os steps devido ele executar o teste somente no “Step 0” e ignorando os demais que tinham botões com problemas de contraste e inputs sem “label” e “name”;
- Em algumas tentativas de refatoração com problemas relacionados a estilo o “Copilot” sugeria algo que acaba quebrando a visualização na maioria das vezes e dessa forma não confiar 100% no que ele gera;

Dessa forma podemos afirma que o Copilot consegue auxiliar o desenvolvedor com possíveis sugestões que vão resolver algumas coisas do dia a dia, porém notei que em questões que precisem de uma análise mais adequada como em arquitetura e design ele se perde pois ele tenta resolver as questões de uma forma mais técnica que envolve somente o arquivo ou trecho de código que está em evidência naquele momento. Ainda não podemos confiar 100% na sua sugestão, mas ela vai dar um norte de como resolver e com a experiência do desenvolvedor ele vai conseguir notar essas problema relacionados irá organizar de forma manual.

Podemos citar também que alguns problemas mais simples de acessibilidade, como inclusão de título, o Copilot conseguiu resolver com poucas intervenções manuais. Porém foi preciso ser feita uma análise mais adequada conforme a experiência do desenvolvedor para identificar a raiz do problema que não estava na página, mas sim no template (Layout) que foi copiado da de outro projeto na internet para acelerar o desenvolvimento do nosso sistema.

3. Contrastes dos Testes de Geração de Código

Essa seção apresenta as observações de geração de código a partir dos testes com sugestões do ambiente real (Teste 1 - T1) e solicitações de código com acessibilidade (Teste 2 - T2). As experiências e evidências geradas fornecem os desafios e oportunidades para o desenvolvimento de software acessível auxiliado pelo Copilot.

No decorrer do desenvolvimento da aplicação Web com Copilot pude perceber que no cenário T1 ele conseguiu auxiliar o desenvolvimento da aplicação de forma bem intuitiva tanto no modo com autocompletar como utilizando prompt para gerar código através de comandos.

Em alguns momentos em ambos casos tive certa dificuldade para fazer a pergunta de forma que a ferramenta fosse mais assertiva em sua resposta, tiveram momentos tanto no cenário T1 quanto no T2 de códigos gerados que não puderam ser reaproveitados e também olhando para o contexto de Front-end web outras que puderam ser reaproveitados.

No caso da acessibilidade no cenário T1 pude perceber que foi gerado em alguns cenários códigos com critérios de acessibilidade, como no caso da utilização de imagens com a tag “”, o copilot gerou o componente HTML já com a propriedade “alt”.

No cenário T1 para geração utilizando “<iframe>” para mapa na tela Sobre, pudemos perceber após executar os testes do “pally” que faltou a propriedade title que foi corrigida executando um comando para correção no cenário T2 para o mesmo.

Tivemos também alguns problemas na geração de código utilizando que utilizavam a tag “<input>” nas telas de listagem de considerações na pesquisa, na tela de Perguntas e Respostas tanto para a quantidade de perguntas que deveria ser selecionada quanto na seleção para o “checkbox” de opções de resposta. O problema foi resolvido utilizando comando para incluir as propriedades necessárias de “name” e “id” para o “<input>”, além da necessidade de gerar um label para cada um deles.

Vale ressaltar também sobre o caso problema gerado no cenário T1 sobre o título das páginas, que foi necessário uma análise mais a fundo na execução do cenário T2 e acabei percebendo que a solução seria fazer uma atuação de correção diretamente no componente template de “Layout”. Outro ponto importante para essa correção que foi identificado foi um problema de contraste gerado em todas as telas no cenário T1, onde pudemos identificar e fazer a correção no diretamente no componente de menu “Sidebar” alterando a cor recomendada pelos critérios de acessibilidade, as outras telas também sofreram com o mesmo problema em alguns outros campos e mais específicos e a solução foi copiada de forma manual.

Tirei de aprendizado com esses testes que nem tudo o Copilot consegue resolver e grande parte dos ajustes vão precisar ser feitos de forma manual, o desenvolvedor ainda vai ter um trabalho grande de pensar em como adaptar o seu dia a dia de desenvolvimento com essas sugestões e aplicar com sabedoria. Dessa forma, podemos afirmar que, essas ferramentas conseguem dar apoio aos desenvolvedores no dia a dia de uma forma que vai melhorar a produtividade. Porém, elas não substituem a experiência desses desenvolvedores no quesito padrões de projetos e padrões de design em um aplicação web, de certa forma o Copilot ajudou bastante com a geração de código porém tiveram momentos que precisei analisar e adaptar a implementação sugerida.